

Last updated on **May 11, 2026**

Deploy models using Model Serving

This article describes Model Serving, the Databricks solution for deploying AI and ML models for real-time serving and batch inference.

What is Model Serving?

Model Serving provides a unified interface to deploy, govern, and query AI models for real-time and batch inference. Each model you serve is available as a REST API that you can integrate into your web or client application.

Model Serving provides a highly available and low-latency service for deploying models. The service automatically scales up or down to meet demand changes, saving infrastructure costs while optimizing latency performance. This functionality uses [serverless compute](#). See the [Model Serving pricing page](#) for more details.

Model Serving offers a unified REST API and MLflow Deployment API for CRUD and querying tasks. In addition, it provides a single UI to manage all your models and their respective serving endpoints. You can also access models directly from SQL using [AI Functions](#) for easy integration into analytics workflows.

AI Functions and Model Serving are tightly integrated for batch inference scenarios. You can use any of the task-specific AI Functions or `ai-query` in your batch inference pipelines. If you choose to use a pre-provisioned model that is hosted and managed by Databricks, you don't need to configure a model serving endpoint yourself.

See the following guides to get started:

- For performing batch inference, see [Enrich data using AI Functions](#).
- For an introductory tutorial on how to serve custom models on Databricks for real-time inference, see [Tutorial: Deploy and query a custom model](#).

 Ask Genie

- For a getting started tutorial on how to query a foundation model on Databricks for real-time inference, see [Get started querying LLMs on Databricks](#).

Models you can deploy

Model serving supports real time and batch inference for the following model types:

- [Custom models](#). These are Python models packaged in the MLflow format. They can be registered either in Unity Catalog or in the workspace model registry. Examples include scikit-learn, XGBoost, PyTorch, and Hugging Face transformer models.
 - Agent serving is supported as a custom model. See [Deploy an agent for generative AI applications \(Model Serving\)](#)
- [Foundation models](#).
 - **Databricks-hosted foundation models** like Meta Llama. These models are available using [Foundation Model APIs](#). These models are curated foundation model architectures that support optimized inference. Base models, like Meta-Llama-3.3-70B-Instruct, GTE-Large, and Mistral-7B are available for immediate use with **pay-per-token** pricing, and workloads that require performance guarantees and fine-tuned model variants can be deployed with **provisioned throughput**.
 - **Foundation models hosted outside of Databricks** like GPT-4 from OpenAI. These models are accessible using [External models](#). The endpoints that serve these models can be centrally governed from Databricks, so you can streamline the use and management of various LLM providers, such as OpenAI and Anthropic, within your organization.

NOTE

You can interact with supported large language models using the [AI Playground](#). The AI Playground is a chat-like environment where you can test, prompt, and compare LLMs. This functionality is available in your Databricks workspace.

Why use Model Serving?

- **Deploy and query any models:** Model Serving provides a unified interface that so you can manage all models in one location and query them with a single API, regardless of whether they are hosted on Databricks or externally. This approach simplifies the process of experimenting with, customizing, and deploying models in production across various clouds and providers.
- **Securely customize models with your private data:** Built on a Data Intelligence Platform, Model Serving simplifies the integration of features and embeddings into models through native integration with the [Databricks Feature Store](#) and [Vector Search](#). For even more improved accuracy and contextual understanding, models can be fine-tuned with proprietary data and deployed effortlessly on Model Serving.
- **Govern and monitor models:** The Serving UI allows you to centrally manage all model endpoints in one place, including those that are externally hosted. You can manage permissions, track and set usage limits and monitor the quality of all types of models using [AI Gateway](#). This enables you to democratize access to SaaS and open LLMs within your organization while ensuring appropriate guardrails are in place.
- **Reduce cost with optimized inference and fast scaling:** Databricks has implemented a range of optimizations to ensure you get the best throughput and latency for large models. The endpoints automatically scale up or down to meet demand changes, saving infrastructure costs while optimizing latency performance. [Monitor model serving costs](#).
 - For workloads that are latency sensitive or involve a high number of queries per second, see [Optimize Model Serving endpoints for production](#) for comprehensive optimization strategies. Reach out to your Databricks account team to ensure your workspace is enabled for high scalability.
- **Bring reliability and security to Model Serving:** Model Serving is designed for high-availability, low-latency production use and can support over 25K queries per second with an overhead latency of less than 50 ms. The serving workloads are protected by multiple layers of security, ensuring a secure and reliable environment for even the most sensitive tasks. You can control network access to Model Serving endpoints by configuring network policies. See [Manage network policies for serverless egress control](#).

 NOTE

Model Serving does not provide security patches to existing model images because of the risk of destabilization to production deployments. A new model image created from a new model version will contain the latest patches. Reach out to your Databricks account team for more information.

Requirements

- Registered model in [Unity Catalog](#) or the [Workspace Model Registry](#).
- Permissions on the registered models as described in [Serving endpoint ACLs](#).
- MLflow 1.29 or higher.
- Workspace entitlements configured. See [Manage entitlements](#).

Enable Model Serving for your workspace

To use Model Serving, your account admin must read and accept the terms and conditions for enabling serverless compute in the account console.

NOTE

If your account was created after March 28, 2022, serverless compute is enabled by default for your workspaces.

If you are not an account admin, you cannot perform these steps. Contact an account admin if your workspace needs access to serverless compute.

1. As an account admin, go to the [feature enablement tab of the account console settings page](#).
2. A banner at the top of the page prompts you to accept the additional terms. Once you read the terms, click **Accept**. If you do not see the banner asking you to accept the terms, this step has been completed already.

After you've accepted the terms, your account is enabled for serverless.

No additional steps are required to enable Model Serving in your workspace.  Ask Genie

Limitations and region availability

Model Serving imposes default limits to ensure reliable performance. See [Model Serving limits and regions](#). If you have feedback on these limits or an endpoint in an unsupported region, reach out to your Databricks account team.

Data protection in Model Serving

Databricks takes data security seriously. Databricks understands the importance of the data you analyze using Model Serving, and implements the following security controls to protect your data.

- Every customer request to Model Serving is logically isolated, authenticated, and authorized.
- Model Serving encrypts all data at rest (AES-256) and in transit (TLS 1.2+).

For all paid accounts, Model Serving does not use user inputs submitted to the service or outputs from the service to train any models or improve any Databricks services.

For all Model Serving workloads, Databricks retains container build logs for up to thirty (30) days and metrics data for up to fourteen (14) days.

For Databricks Foundation Model APIs, as part of providing the service, Databricks may temporarily process and store inputs and outputs for the purposes of preventing, detecting, and mitigating abuse or harmful uses. Your inputs and outputs are isolated from those of other customers, stored in the same region as your workspace for up to thirty (30) days, and only accessible for detecting and responding to security or abuse concerns.

Foundation Model APIs is a [Databricks Designated Service](#), meaning it adheres to data residency boundaries as implemented by [Databricks Geos](#).

Additional resources

- [Get started querying LLMs on Databricks](#).
- [Tutorial: Deploy and query a custom model](#)

- [Tutorial: Create external model endpoints to query OpenAI models](#)
- [Build gen AI apps on Databricks](#)
- [Enrich data using AI Functions](#)
- [Migrate to Model Serving](#)

Is this page

as this page helpful?

☐ Yes

☐ No



Last updated on **May 11, 2026**

Model Serving concepts

This page provides definitions of key concepts that are used in Model Serving for model deployments.

Endpoint

REST API that exposes one or more served models for inference.

Route-optimized endpoint

Endpoint property that enables an improved network path with faster, more direct communication between the user and the model during inference. For more information, see [Route optimization on serving endpoints](#).

Provisioned concurrency

Endpoint property that specifies the maximum number of parallel requests an endpoint can handle. Estimate the required concurrency using the formula:

```
provisioned concurrency = queries per second (QPS) \* model execution time (s).
```

Scale to zero

Endpoint property that automatically reduces resource consumption to zero when endpoint is not in-use. Scale to zero is recommended for testing and development. However, scale to zero is not recommended for production endpoints, as latency is greater and capacity is not guaranteed when scaled to zero.



Served entity

Named deployment unit inside an endpoint that represents a specific model with its compute configuration that can receive routed traffic.

Traffic configuration

Specification for what percentage of traffic to an endpoint should go to each model. Traffic configuration is required for endpoints with more than one served model.

The following is an example where the endpoint named `multi-pt-model` hosts version 2 of `meta_llama_v3_1_70b_instruct` which gets 60% of the endpoint traffic, and also hosts version 3 of `meta_llama_v3_1_8b_instruct` which gets 40% of the endpoint traffic. For more information, see [Serve multiple models to a model serving endpoint](#).

Bash

```
POST /api/2.0/serving-endpoints
```

```
{
  "name": "multi-pt-model"
  "config":
  {
    "served_entities":
    [
      {
        "name": "meta_llama_v3_1_70b_instruct",
        "entity_name": "system.ai.meta_llama_v3_1_70b_instruct",
        "entity_version": "4",
        "min_provisioned_throughput": 0,
        "max_provisioned_throughput": 2400
      },
      {
        "name": "meta_llama_v3_1_8b_instruct",
        "entity_name": "system.ai.meta_llama_v3_1_8b_instruct",
        "entity_version": "4",
        "min_provisioned_throughput": 0,
        "max_provisioned_throughput": 1240
      }
    ],
    "traffic_config":
    {
      "routes":
```

```
[
  {
    "served_model_name":"meta_llama_v3_1_8b_instruct",
    "traffic_percentage":"60"
  },
  {
    "served_model_name":"meta_llama_v3_1_70b_instruct",
    "traffic_percentage":"40"
  }
]
```

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 12, 2026**

Supported foundation models on Model Serving

This article describes the foundation models you can serve using [Model Serving](#).

Foundation models are large, pre-trained neural networks that are trained on both large and broad ranges of data. These models are designed to learn general patterns in language, images, or other data types, and can be fine-tuned for specific tasks with additional training. Your use of certain foundation models is subject to the model's terms and acceptable use policy. See [Applicable model terms](#).

Model Serving offers flexible options for hosting and querying foundation models based on your needs:

- [Pay-per-token](#): Ideal for experimentation and quick exploration. This option allows you to query pre-configured endpoints in your Databricks workspace without upfront infrastructure commitments.
- [AI Functions \(batch inference\)](#): A subset of Databricks-hosted models are optimized for AI Functions. You can apply AI to your data and run batch inference production workloads at scale using these functions and their supported models.
- [Provisioned throughput](#): Recommended for production use cases requiring performance guarantees. This option enables the deployment of fine-tuned foundation models with optimized serving endpoints.
- [External models](#): This option enables access to foundation models hosted outside of Databricks, such as those provided by OpenAI or Anthropic. These models can be centrally managed within Databricks for streamlined governance.

Foundation models hosted on Databricks

Databricks hosts state-of-the-art open foundation models. These models are made available using [Foundation Model APIs](#).

The following table summarizes which Databricks-hosted models and model families are supported in each region based on the Model Serving feature.

❗ IMPORTANT

- Google Gemini 3 Flash and Google Gemini 3 Pro are hosted on global endpoints and require [cross geography routing to be enabled](#) for every region. Google Gemini 3 Pro will be retired on March 26, 2026. See [Retired models](#) for the recommended replacement model and guidance for how to migrate during deprecation.
 - To allow more time for migration, between March 26, 2026 and June 7, 2026, API calls to Gemini 3 Pro will be temporarily redirected to Gemini 3.1 Pro. The pricing for both models is identical.
- OpenAI GPT-5.1 Codex Max and OpenAI GPT-5.1 Codex Mini are hosted on global endpoints and require [cross geography routing to be enabled](#) for every region.
- OpenAI GPT-5.1 Codex Max, OpenAI GPT-5.1 Codex Mini, and OpenAI GPT-5.2 Codex will be retired on July 16, 2026. See [Retired models](#) for the recommended replacement model and guidance for how to migrate during deprecation.
- Anthropic Claude 3.7 Sonnet is no longer available. See [Retired models](#) for the recommended replacement model and guidance for how to migrate during deprecation.
- Meta Llama 4 Maverick is available for Foundation Model APIs provisioned throughput workloads in [Public Preview](#).
- Meta-Llama-3.1-405B-Instruct is no longer available for pay-per-token workloads. Starting May 15, 2026, it will also be retired for provisioned throughput workloads. See [Retired models](#) for the recommended replacement model and guidance for how to migrate during deprecation.
- Several older model families have been retired. See [Retired models](#) for the full list of retired models and recommended replacements.

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
ap-northeast-1	The following models are supported: databricks-gpt-5-5-pro ↗ databricks-gpt-5-5 ↗ databricks-gpt-5-4 ↗ databricks-gpt-5-4-mini ↗ databricks-gpt-5-4-nano ↗ databricks-gpt-5-3-codex ↗ databricks-gpt-5-2-codex ↗ databricks-gpt-5-2 ↗ databricks-gpt-5-1 ↗ databricks-gpt-5-1-codex-max ↗ databricks-gpt-5-1-codex-mini ↗	The following models are supported: databricks-gpt-5-2 ↗ databricks-gpt-5-1 ↗ databricks-gpt-5 ↗ databricks-gpt-5-mini ↗ databricks-gpt-5-nano ↗ databricks-gemini-3-1-pro ↗ databricks-gemini-3-pro ↗ databricks-gemini-3-flash ↗ databricks-gemini-2-5-pro ↗ databricks-gemini-2-5-flash ↗ databricks-qwen35-122b-a10b ↗	- The following model families are supported for real-time inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B↗ - Alibaba Cloud Qwen3.5 122B A10B (preview) - Meta Llama 4 Maverick↗ (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	<code>databricks-gpt-5</code> ↗ <code>databricks-gpt-5-mini</code> ↗ <code>databricks-gpt-5-nano</code> ↗ <code>databricks-gemini-3-1-pro</code> ↗ <code>databricks-gemini-3-pro</code> ↗ <code>databricks-gemini-3-1-flash-lite</code> ↗ <code>databricks-gemini-3-flash</code> ↗ <code>databricks-gemini-2-5-pro</code> ↗ <code>databricks-gemini-2-5-flash</code> ↗ <code>databricks-qwen3-122b-a10b</code> <code>databricks-qwen3-next-80b-a3b-instruct</code>	<code>databricks-qwen3-next-80b-a3b-instruct</code> <code>databricks-claude-sonnet-4</code> <code>databricks-gpt-oss-20b</code> <code>databricks-gpt-oss-120b</code> <code>databricks-gemma-3-12b</code> ↗ <code>databricks-llama-4-maverick</code> ↗ <code>databricks-meta-llama-3-3-70b-instruct</code> ↗ <code>databricks-meta-llama-3-1-8b-instruct</code> ↗ <code>databricks-qwen3-embedding-0-6b</code> ↗ <code>databricks-gte-large-en</code> ↗	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks- claude- sonnet-4-6 • databricks- claude- sonnet-4-5 • databricks- claude-haiku- 4-5 • databricks- claude-opus- 4-7 • databricks- claude-opus- 4-6 ↔ • databricks- claude-opus- 4-5 ↔ • databricks- claude-opus- 4-1 ↔ • databricks- gpt-oss-20b • databricks- gpt-oss-120b • databricks- gemma-3-12b ↔ • databricks- claude- sonnet-4 ↔ • databricks- llama-4-		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	maverick ↗ • databricks-meta-llama-3-3-70b-instruct ↗ • databricks-meta-llama-3-1-405b-instruct ↗ • databricks-meta-llama-3-1-8b-instruct ↗ • databricks-qwen3-embedding-0-6b ↗ • databricks-gte-large-en ↗		
ap-northeast-2	The following models are supported: • databricks-gpt-5-5-pro ↗ • databricks-gpt-5-5 ↗ • databricks-gpt-5-4 ↗	The following models are supported: • databricks-gpt-5-2 ↗ • databricks-gpt-5-1 ↗ • databricks-gpt-5 ↗	• The following model families are supported for real-time inference: ◦ OpenAI GPT OSS 120B ◦ OpenAI GPT OSS 20B ◦ Google Gemini 3 Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-5-4-mini ↗ • databricks-gpt-5-4-nano ↗ • databricks-gpt-5-3-codex ↗ • databricks-gpt-5-2-codex ↗ • databricks-gpt-5-2 ↗ • databricks-gpt-5-1 ↗ • databricks-gpt-5-1-codex-max ↗ • databricks-gpt-5-1-codex-mini ↗ • databricks-gpt-5 ↗ • databricks-gpt-5-mini ↗ • databricks-gpt-5-nano ↗ • databricks-gemini-3-1-pro ↗	• databricks-gpt-5-mini ↗ • databricks-gpt-5-nano ↗ • databricks-gemini-3-1-pro ↗ • databricks-gemini-3-pro ↗ • databricks-gemini-3-flash ↗ • databricks-gemini-2-5-pro ↗ • databricks-gemini-2-5-flash ↗ • databricks-qwen3-next-80b-a3b-instruct ↗ • databricks-claude-sonnet-4 • databricks-gpt-oss-20b • databricks-gpt-oss-120b • databricks-gemma-3-12b ↗	12B ↗ ◦ Meta Llama 4 Maverick ↗ (preview) ◦ Meta Llama 3.3 ◦ Meta Llama 3.2 3B ◦ Meta Llama 3.2 1B ◦ Meta Llama 3.1 ◦ GTE v1.5 (English) ◦ BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-pro ↔ - databricks-gemini-3-1-flash-lite ↔ - databricks-gemini-3-flash ↔ - databricks-gemini-2-5-pro ↔ - databricks-gemini-2-5-flash ↔ - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-7 ↔ - databricks-claude-opus-4-6 ↔ - databricks-claude-opus-	- databricks-llama-4-maverick ↔ - databricks-meta-llama-3-3-70b-instruct ↔ - databricks-meta-llama-3-1-8b-instruct ↔ - databricks-qwen3-embedding-0-6b ↔ - databricks-gte-large-en ↔	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	4-5 ↔ - databricks-claude-opus-4-1 ↔ - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b ↔ - databricks-qwen3-next-80b-a3b-instruct ↔ - databricks-claude-sonnet-4 ↔ - databricks-llama-4-maverick ↔ - databricks-meta-llama-3-3-70b-instruct ↔ - databricks-meta-llama-3-1-405b-instruct ↔ - databricks-meta-llama-3-1-8b-instruct ↔		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	databricks-qwen3-embedding-0-6b ↗ databricks-gte-large-en ↗		
ap-south-1	The following models are supported: databricks-gpt-5-5-pro ↗ databricks-gpt-5-5 ↗ databricks-gpt-5-4 ↗ databricks-gpt-5-4-mini ↗ databricks-gpt-5-4-nano ↗ databricks-gpt-5-3-codex ↗ databricks-gpt-5-2-codex ↗ databricks-gpt-5-2 ↗	The following models are supported: databricks-gpt-5-2 ↗ databricks-gpt-5-1 ↗ databricks-gpt-5 ↗ databricks-gpt-5-mini ↗ databricks-gpt-5-nano ↗ databricks-gemini-3-1-pro ↗ databricks-gemini-3-pro ↗ databricks-gemini-3-flash ↗	- The following model families are supported for real-time inference: - OpenAI GPT OSS 120B ↗ - OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B ↗ - Meta Llama 4 Maverick ↗ (preview) - Meta Llama 3.3 ↗ - Meta Llama 3.2 3B ↗ - Meta Llama 3.2 1B ↗ - Meta Llama 3.1 ↗  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-5-1 ↗ • databricks-gpt-5-1-codex-max ↗ • databricks-gpt-5-1-codex-mini ↗ • databricks-gpt-5 ↗ • databricks-gpt-5-mini ↗ • databricks-gpt-5-nano ↗ • databricks-gemini-3-1-pro ↗ • databricks-gemini-3-pro ↗ • databricks-gemini-3-1-flash-lite ↗ • databricks-gemini-3-flash ↗ • databricks-gemini-2-5-pro ↗ • databricks-gemini-2-5-flash ↗	• databricks-gemini-2-5-pro ↗ • databricks-gemini-2-5-flash ↗ • databricks-qwen3-next-80b-a3b-instruct ↗ • databricks-claude-sonnet-4 • databricks-gpt-oss-20b ↗ • databricks-gpt-oss-120b ↗ • databricks-gemma-3-12b ↗ • databricks-llama-4-maverick ↗ • databricks-meta-llama-3-3-70b-instruct ↗ • databricks-meta-llama-3-1-8b-instruct ↗	◦ GTE v1.5 (English) ◦ BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks- claude- sonnet-4-6 ↔ - databricks- claude- sonnet-4-5 ↔ - databricks- claude-haiku- 4-5 ↔ - databricks- claude-opus- 4-7 ↔ - databricks- claude-opus- 4-6 ↔ - databricks- claude-opus- 4-5 ↔ - databricks- claude-opus- 4-1 ↔ - databricks- gpt-oss-20b ↔ - databricks- gpt-oss- 120b ↔ - databricks- gemma-3-12b ↔ - databricks- qwen3-next- 80b-a3b- instruct ↔	- databricks- qwen3- embedding-0- 6b ↔ - databricks- gte-large- en ↔	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks- claude- sonnet-4 ↔ - databricks- llama-4- maverick ↔ - databricks- meta-llama-3- 3-70b- instruct ↔ - databricks- meta-llama-3- 1-405b- instruct ↔ - databricks- meta-llama-3- 1-8b- instruct ↔ - databricks- qwen3- embedding-0- 6b ↔ - databricks- gte-large- en ↔		
ap- southeast- 1	The following models are supported: databricks- gpt-5-5-pro ↔	The following models are supported: databricks- gpt-5-2 ↔	The following model families are supported and optimized for batch inference:  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-5-5 ↗ • databricks-gpt-5-4 ↗ • databricks-gpt-5-4-mini ↗ • databricks-gpt-5-4-nano ↗ • databricks-gpt-5-3-codex ↗ • databricks-gpt-5-2-codex ↗ • databricks-gpt-5-2 ↗ • databricks-gpt-5-1 ↗ • databricks-gpt-5-1-codex-max ↗ • databricks-gpt-5-1-codex-mini ↗ • databricks-gpt-5 ↗ • databricks-gpt-5-mini ↗ • databricks-gpt-5-nano ↗	• databricks-gpt-5-1 ↗ • databricks-gpt-5 ↗ • databricks-gpt-5-mini ↗ • databricks-gpt-5-nano ↗ • databricks-gemini-3-1-pro ↗ • databricks-gemini-3-pro ↗ • databricks-gemini-3-flash ↗ • databricks-gemini-2-5-pro ↗ • databricks-gemini-2-5-flash ↗ • databricks-qwen3-next-80b-a3b-instruct ↗ • databricks-claude-sonnet-4 ↗ • databricks-gpt-oss-20b ↗	◦ OpenAI GPT OSS 120B ◦ OpenAI GPT OSS 20B ◦ Google Gemma 3 12B ↗ ◦ Meta Llama 4 Maverick ↗ (preview) • The following model families are supported for real-time inference: ◦ OpenAI GPT OSS 120B ◦ OpenAI GPT OSS 20B ◦ Google Gemma 3 12B ↗ ◦ Meta Llama 4 Maverick ↗ (preview) ◦ Meta Llama 3.3 ◦ Meta Llama 3.2 3B ◦ Meta Llama 3.2 1B

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-1-flash-lite ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro ↗ - databricks-gemini-2-5-flash ↗ - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-7 - databricks-claude-opus-	- databricks-gpt-oss-120b - databricks-gemma-3-12b ↗ - databricks-llama-4-maverick ↗ - databricks-meta-llama-3-3-70b-instruct ↗ - databricks-meta-llama-3-1-8b-instruct ↗ - databricks-qwen3-embedding-0-6b - databricks-gte-large-en	- Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	4-6 ↔ - databricks-claude-opus-4-5 ↔ - databricks-claude-opus-4-1 ↔ - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b ↔ - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 ↔ - databricks-llama-4-maverick ↔ - databricks-meta-llama-3-3-70b-instruct ↔ - databricks-meta-llama-3-1-405b-instruct ↔		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	<code>databricks-meta-llama-3-1-8b-instruct</code> ↗ <code>databricks-qwen3-embedding-0-6b</code> ↗ <code>databricks-gte-large-en</code> ↗		
ap-southeast-2	The following models are supported: <code>databricks-gpt-5-5-pro</code> ↗ <code>databricks-gpt-5-5</code> ↗ <code>databricks-gpt-5-4</code> ↗ <code>databricks-gpt-5-4-mini</code> ↗ <code>databricks-gpt-5-4-nano</code> ↗ <code>databricks-gpt-5-3-codex</code> ↗	The following models are supported: <code>databricks-gpt-5-2</code> ↗ <code>databricks-gpt-5-1</code> ↗ <code>databricks-gpt-5</code> ↗ <code>databricks-gpt-5-mini</code> ↗ <code>databricks-gpt-5-nano</code> ↗ <code>databricks-gemini-3-1-pro</code> ↗ <code>databricks-gemini-3-pro</code> ↗	- The following model families are supported and optimized for batch inference: - OpenAI GPT OSS 120B ↗ - OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B ↗ - Meta Llama 4 Maverick ↗ (preview) - The following model families are supported for  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gpt-5-2-codex ↗ - databricks-gpt-5-2 ↗ - databricks-claude-opus-4-6 - databricks-claude-opus-4-5 ↗ - databricks-gpt-5-1 ↗ - databricks-gpt-5-1-codex-max ↗ - databricks-gpt-5-1-codex-mini ↗ - databricks-gpt-5 ↗ - databricks-gpt-5-mini ↗ - databricks-gpt-5-nano ↗ - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗	- databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro ↗ - databricks-gemini-2-5-flash ↗ - databricks-qwen3-next-80b-a3b-instruct ↗ - databricks-claude-sonnet-4 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b ↗ - databricks-qwen35-122b-a10b - databricks-llama-4-maverick ↗ - databricks-meta-llama-3-3-70b-instruct ↗	real-time inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B ↗ - Alibaba Cloud Qwen3.5 122B A10B (preview) - Meta Llama 4 Maverick ↗ (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-1-flash-lite↔ - databricks-gemini-3-flash↔ - databricks-gemini-2-5-pro↔ - databricks-gemini-2-5-flash↔ - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-7↔ - databricks-claude-opus-4-1↔ - databricks-gpt-oss-20b - databricks-gpt-oss-120b	- databricks-meta-llama-3-1-8b-instruct↔ - databricks-qwen3-embedding-0-6b - databricks-gte-large-en	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gemma-3-12b↔ • databricks-qwen35-122b-a10b • databricks-qwen3-next-80b-a3b-instruct↔ • databricks-claude-sonnet-4↔ • databricks-llama-4-maverick↔ • databricks-meta-llama-3-3-70b-instruct↔ • databricks-meta-llama-3-1-405b-instruct↔ • databricks-meta-llama-3-1-8b-instruct↔ • databricks-qwen3-embedding-0-6b		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	<code>databricks-gte-large-en</code>		
ca-central-1	The following models are supported: <code>databricks-gpt-5-5-pro</code> ↗ <code>databricks-gpt-5-5</code> ↗ <code>databricks-gpt-5-4</code> ↗ <code>databricks-gpt-5-4-mini</code> ↗ <code>databricks-gpt-5-4-nano</code> ↗ <code>databricks-gpt-5-3-codex</code> ↗ <code>databricks-gpt-5-2-codex</code> ↗ <code>databricks-gpt-5-2</code> ↗ <code>databricks-claude-opus-4-6</code> ↗ <code>databricks-claude-opus-</code>	The following models are supported: <code>databricks-gpt-5-2</code> ↗ <code>databricks-gpt-5-1</code> ↗ <code>databricks-gpt-5</code> ↗ <code>databricks-gpt-5-mini</code> ↗ <code>databricks-gpt-5-nano</code> ↗ <code>databricks-gemini-3-1-pro</code> ↗ <code>databricks-gemini-3-pro</code> ↗ <code>databricks-gemini-3-flash</code> ↗ <code>databricks-gemini-2-5-pro</code> ↗ <code>databricks-gemini-2-5-flash</code> ↗	- The following model families are supported and optimized for batch inference: - OpenAI GPT OSS 120B ↗ - OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - The following model families are supported for real-time inference: - OpenAI GPT OSS 120B ↗ - OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B - Meta Llama 4  Ask Genie Maverick

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	4-5 ↗ - databricks-gpt-5-1 ↗ - databricks-gpt-5-1-codex-max ↗ - databricks-gpt-5-1-codex-mini ↗ - databricks-gpt-5 ↗ - databricks-gpt-5-mini ↗ - databricks-gpt-5-nano ↗ - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-1-flash-lite ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro ↗ - databricks-gemini-2-5-	- databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b ↗ - databricks-gpt-oss-120b ↗ - databricks-gemma-3-12b ↗ - databricks-llama-4-maverick ↗ - databricks-meta-llama-3-3-70b-instruct ↗ - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b ↗ - databricks-gte-large-en ↗	(preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	flash ↗ • databricks- claude- sonnet-4-6 ↗ • databricks- claude- sonnet-4-5 ↗ • databricks- claude-haiku- 4-5 ↗ • databricks- claude-opus- 4-7 • databricks- claude-opus- 4-1 ↗ • databricks- gpt-oss-20b ↗ • databricks- gpt-oss- 120b ↗ • databricks- gemma-3-12b ↗ • databricks- qwen3-next- 80b-a3b- instruct • databricks- claude- sonnet-4 ↗ • databricks- llama-4-		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	maverick ↗ - databricks-meta-llama-3-3-70b-instruct ↗ - databricks-meta-llama-3-1-405b-instruct ↗ - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en		
eu-central-1	The following models are supported: - databricks-gpt-5-5-pro - databricks-gpt-5-5 - databricks-gpt-5-4 - databricks-gpt-5-4-mini	The following models are supported: - databricks-gpt-5-2 ↗ - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini	- The following model families are supported and optimized for batch inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google  Ask Genie Gemma 3 12B

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gpt-5-4-nano - databricks-gpt-5-2 ↔ - databricks-claude-opus-4-6 ↔ - databricks-claude-opus-4-5 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro ↔ - databricks-gemini-3-pro ↔ - databricks-gemini-3-1-flash-lite ↔ - databricks-gemini-3-flash ↔ - databricks-gemini-2-5-	- databricks-gpt-5-nano - databricks-gemini-3-1-pro ↔ - databricks-gemini-3-pro ↔ - databricks-gemini-3-flash ↔ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-qwen35-122b-	- Meta Llama 4 Maverick (preview) - The following model families are supported for real-time inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B - Alibaba Cloud Qwen3.5 122B A10B (preview) - Meta Llama 4 Maverick (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	pro databricks-gemini-2-5-flash databricks- claude-sonnet-4-6 databricks- claude-sonnet-4-5 databricks- claude-haiku-4-5 databricks- claude-opus-4-7 databricks- claude-opus-4-1 databricks-gpt-oss-20b databricks-gpt-oss-120b databricks-gemma-3-12b databricks-qwen35-122b-a10b databricks- llama-4-maverick	a10b databricks- llama-4-maverick databricks- meta-llama-3-3-70b-instruct databricks- meta-llama-3-1-8b-instruct databricks-qwen3-embedding-0-6b databricks-gte-large-en	BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	<code>databricks- claude- sonnet-4</code> <code>databricks- meta-llama-3- 3-70b- instruct</code> <code>databricks- meta-llama-3- 1-405b- instruct</code> ↔ <code>databricks- meta-llama-3- 1-8b- instruct</code> <code>databricks- qwen3- embedding-0- 6b</code> <code>databricks- gte-large-en</code> <code>databricks- qwen3-next- 80b-a3b- instruct</code>		
<code>eu-west-1</code>	The following models are supported: <code>databricks- gpt-5-5-pro</code>	The following models are supported: <code>databricks- gpt-5-2</code> ↔	The following model families are supported and optimized for batch inference:  Ask Genie

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-5-5 • databricks-gpt-5-4 • databricks-gpt-5-4-mini • databricks-gpt-5-4-nano • databricks-gpt-5-2 ↔ • databricks-claude-opus-4-6 ↔ • databricks-claude-opus-4-5 • databricks-gpt-5-1 • databricks-gpt-5 • databricks-gpt-5-mini • databricks-gpt-5-nano • databricks-gemini-3-1-pro ↔ • databricks-gemini-3-pro ↔	• databricks-gpt-5-1 • databricks-gpt-5 • databricks-gpt-5-mini • databricks-gpt-5-nano • databricks-gemini-3-1-pro ↔ • databricks-gemini-3-pro ↔ • databricks-gemini-3-flash ↔ • databricks-gemini-2-5-pro • databricks-gemini-2-5-flash • databricks-qwen3-next-80b-a3b-instruct • databricks-claude-sonnet-4 • databricks-gpt-oss-20b	◦ OpenAI GPT OSS 120B ◦ OpenAI GPT OSS 20B ◦ Google Gemma 3 12B ◦ Meta Llama 4 Maverick (preview) • The following model families are supported for real-time inference: ◦ OpenAI GPT OSS 120B ◦ OpenAI GPT OSS 20B ◦ Google Gemma 3 12B ◦ Alibaba Cloud Qwen3.5 122B A10B (preview) ◦ Meta Llama 4 Maverick (preview) ◦ Meta Llama 3.3 ◦ Meta Llama 3.2 3B

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-1-flash-lite↔ - databricks-gemini-3-flash↔ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-7 - databricks-claude-opus-4-1 - databricks-gpt-oss-20b - databricks-gpt-oss-120b	- databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-qwen35-122b-a10b - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en	- Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemma-3-12b - databricks-qwen3-122b-a10b - databricks-claude-sonnet-4 - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-405b-instruct↔ - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en - databricks-qwen3-next-		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	80b-a3b-instruct		
eu-west-2	The following models are supported: - databricks-gpt-5-5-pro - databricks-gpt-5-5 - databricks-gpt-5-4 - databricks-gpt-5-4-mini - databricks-gpt-5-4-nano - databricks-gpt-5-2 ↗ - databricks-claude-opus-4-6 ↗ - databricks-claude-opus-4-5 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini	The following models are supported: - databricks-gpt-5-2 ↗ - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash	- The following model families are supported and optimized for batch inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - The following model families are supported for real-time inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B - Meta Llama 4 Ask Genie Maverick

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-5-nano • databricks-gemini-3-1-pro ↗ • databricks-gemini-3-pro ↗ • databricks-gemini-3-1-flash-lite ↗ • databricks-gemini-3-flash ↗ • databricks-gemini-2-5-pro • databricks-gemini-2-5-flash • databricks-claude-sonnet-4-6 • databricks-claude-sonnet-4-5 • databricks-claude-haiku-4-5 • databricks-claude-opus-4-7	• databricks-qwen3-next-80b-a3b-instruct • databricks-claude-sonnet-4 • databricks-gpt-oss-20b • databricks-gpt-oss-120b • databricks-gemma-3-12b • databricks-llama-4-maverick • databricks-meta-llama-3-3-70b-instruct • databricks-meta-llama-3-1-8b-instruct • databricks-qwen3-embedding-0-6b • databricks-gte-large-en	(preview) ◦ Meta Llama 3.3 ◦ Meta Llama 3.2 3B ◦ Meta Llama 3.2 1B ◦ Meta Llama 3.1 ◦ GTE v1.5 (English) ◦ BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-claude-opus-4-1 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-claude-sonnet-4 - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-405b-instruct ↗ - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	<code>databricks-gte-large-en</code> <code>databricks-qwen3-next-80b-a3b-instruct</code>		
<code>eu-west-3</code>	Not supported	Not supported	Not supported
<code>sa-east-1</code>	The following models are supported: <code>databricks-gpt-5-5-pro</code> ↗ <code>databricks-gpt-5-5</code> ↗ <code>databricks-gpt-5-4</code> ↗ <code>databricks-gpt-5-4-mini</code> ↗ <code>databricks-gpt-5-4-nano</code> ↗ <code>databricks-gpt-5-3-codex</code> ↗ <code>databricks-gpt-5-2-codex</code> ↗ <code>databricks-gpt-5-2</code> ↗	The following models are supported: <code>databricks-gpt-5-2</code> ↗ <code>databricks-gpt-5-1</code> ↗ <code>databricks-gpt-5</code> ↗ <code>databricks-gpt-5-mini</code> ↗ <code>databricks-gpt-5-nano</code> ↗ <code>databricks-gemini-3-1-pro</code> ↗ <code>databricks-gemini-3-pro</code> ↗ <code>databricks-gemini-3-flash</code> ↗	- The following model families are supported and optimized for batch inference: - OpenAI GPT OSS 120B ↗ - OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - The following model families are supported for real-time inference: - OpenAI GPT OSS 120B ↗

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gpt-5-1 ↗ - databricks-gpt-5-1-codex-max ↗ - databricks-gpt-5-1-codex-mini ↗ - databricks-gpt-5 ↗ - databricks-gpt-5-mini ↗ - databricks-gpt-5-nano ↗ - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-1-flash-lite ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro ↗ - databricks-gemini-2-5-flash ↗	- databricks-gemini-2-5-pro ↗ - databricks-gemini-2-5-flash ↗ - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b ↗ - databricks-gpt-oss-120b ↗ - databricks-gemma-3-12b ↗ - databricks-llama-4-maverick ↗ - databricks-meta-llama-3-3-70b-instruct ↗ - databricks-meta-llama-3-1-8b-instruct	- OpenAI GPT OSS 20B ↗ - Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks- claude- sonnet-4-6 ↔ • databricks- claude- sonnet-4-5 ↔ • databricks- claude-haiku- 4-5 ↔ • databricks- claude-opus- 4-7 • databricks- claude-opus- 4-6 ↔ • databricks- claude-opus- 4-5 ↔ • databricks- claude-opus- 4-1 ↔ • databricks- gpt-oss-20b ↔ • databricks- gpt-oss- 120b ↔ • databricks- gemma-3-12b ↔ • databricks- qwen3-next- 80b-a3b- instruct	• databricks- qwen3- embedding-0- 6b ↔ • databricks- gte-large- en ↔	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks- claude- sonnet-4 ↔ - databricks- llama-4- maverick ↔ - databricks- meta-llama-3- 3-70b- instruct ↔ - databricks- meta-llama-3- 1-405b- instruct ↔ - databricks- meta-llama-3- 1-8b- instruct - databricks- qwen3- embedding-0- 6b ↔ - databricks- gte-large- en ↔		
us-east-1	The following models are supported: databricks- gpt-5-5-pro	The following models are supported: databricks- gpt-5-2	The following model families are supported: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gpt-5-5 - databricks-gpt-5-4 - databricks-gpt-5-4-mini - databricks-gpt-5-4-nano - databricks-gpt-5-2 - databricks-claude-opus-4-7 - databricks-claude-opus-4-6 - databricks-claude-opus-4-5 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro ↗	- databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b	- Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-pro↔ - databricks-gemini-3-1-flash-lite↔ - databricks-gemini-3-flash↔ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-1 - databricks-gpt-oss-20b - databricks-gpt-oss-120b	- databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemma-3-12b - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-405b-instruct - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en		
 Ask Genie			

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
us-east-2	The following models are supported: - databricks-gpt-5-5-pro - databricks-gpt-5-5 - databricks-gpt-5-4 - databricks-gpt-5-4-mini - databricks-gpt-5-4-nano - databricks-gpt-5-2 - databricks-claude-opus-4-7 - databricks-claude-opus-4-6 - databricks-claude-opus-4-5 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini	The following models are supported: - databricks-gpt-5-2 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro ↗ - databricks-gemini-3-pro ↗ - databricks-gemini-3-flash ↗ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen3-next	The following model families are supported for real-time inference: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B - Meta Llama 4 Maverick (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gpt-5-nano - databricks-gemini-3-1-pro ↔ - databricks-gemini-3-pro ↔ - databricks-gemini-3-1-flash-lite ↔ - databricks-gemini-3-flash ↔ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-opus-4-1	80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	• databricks-gpt-oss-20b • databricks-gpt-oss-120b • databricks-gemma-3-12b • databricks-qwen3-next-80b-a3b-instruct • databricks-claude-sonnet-4 • databricks-llama-4-maverick • databricks-meta-llama-3-3-70b-instruct • databricks-meta-llama-3-1-405b-instruct • databricks-meta-llama-3-1-8b-instruct • databricks-qwen3-embedding-0-6b		

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	databricks-gte-large-en		
us-gov-west-1	Not supported	Not supported	Not supported
us-west-1	Not supported	Not supported	Not supported
us-west-2	The following models are supported: databricks-gpt-5-5-pro databricks-gpt-5-5 databricks-gpt-5-4 databricks-gpt-5-4-mini databricks-gpt-5-4-nano databricks-gpt-5-2 databricks-gpt-5-1 databricks-gpt-5 databricks-gpt-5-mini databricks-gpt-5-nano	The following models are supported: databricks-gpt-5-2 databricks-gpt-5-1 databricks-gpt-5 databricks-gpt-5-mini databricks-gpt-5-nano databricks-gemini-3-1-pro ⇨ databricks-gemini-3-pro ⇨ databricks-gemini-3-flash ⇨ databricks-gemini-2-5-	The following model families are supported: - OpenAI GPT OSS 120B - OpenAI GPT OSS 20B - Google Gemma 3 12B - Alibaba Cloud Qwen3.5 122B A10B (preview) - Meta Llama 4 Maverick (preview) - Meta Llama 3.3 - Meta Llama 3.2 3B - Meta Llama 3.2 1B - Meta Llama 3.1 - GTE v1.5 (English) - BGE v1.5 (English)

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-gemini-3-1-pro↔ - databricks-gemini-3-pro↔ - databricks-gemini-3-1-flash-lite↔ - databricks-gemini-3-flash↔ - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen35-122b-a10b - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5	- pro - databricks-gemini-2-5-flash - databricks-qwen35-122b-a10b - databricks-qwen3-next-80b-a3b-instruct - databricks-claude-sonnet-4 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-gemma-3-12b - databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-8b-instruct	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks- claude-haiku-4-5 - databricks- claude-opus-4-7 - databricks- claude-opus-4-6 - databricks- claude-opus-4-5 - databricks- claude-opus-4-1 - databricks- gpt-oss-20b - databricks- gpt-oss-120b - databricks- gemma-3-12b - databricks- claude-sonnet-4 - databricks- llama-4-maverick - databricks- meta-llama-3-3-70b-instruct	- databricks- qwen3-embedding-0-6b - databricks- gte-large-en	

Region	Foundation Model APIs pay-per-token	AI Functions (batch inference)	Foundation Model APIs provisioned throughput
	- databricks-meta-llama-3-1-405b-instruct - databricks-meta-llama-3-1-8b-instruct - databricks-qwen3-embedding-0-6b - databricks-gte-large-en		

⇒ This model is supported based on GPU availability and requires [cross geography routing to be enabled](#).

Access foundation models hosted outside of Databricks

Foundation models created by LLM providers, such as OpenAI and Anthropic, are also accessible on Databricks using [External models](#). These models are hosted outside of Databricks and you can create an endpoint to query them. These endpoints can be centrally governed from Databricks, which streamlines the use and management of various LLM providers within your organization.

The following table presents a non-exhaustive list of supported models and corresponding [endpoint types](#). You can use the listed model associations to help you configure your an endpoint for any newly released model types as they become available with a given provider. Customers are responsible for ensuring compliance with applicable model licenses.

NOTE

With the rapid development of LLMs, there is no guarantee that this list is up to date at all times. New model versions from the same provider are typically supported even if they are not on the list.

Model provider	llm/v1/completions	llm/v1/chat	llm/v1/embeddings
OpenAI**	- gpt-3.5-turbo-instruct - babbage-002 - davinci-002	- o1 - o1-mini - o1-mini-2024-09-12 - gpt-3.5-turbo - gpt-4 - gpt-4-turbo - gpt-4-turbo-2024-04 - gpt-4o - gpt-4o-2024-05-13 - gpt-4o-mini	- text-embedding-ada-002 - text-embedding-3-large - text-embedding-3-small
Azure OpenAI**	- text-davinci-003 - gpt-35-turbo-instruct	- o1 - o1-mini - gpt-35-turbo - gpt-35-turbo-16k - gpt-4 - gpt-4-turbo - gpt-4-32k - gpt-4o - gpt-4o-mini	- text-embedding-ada-002 - text-embedding-3-large - text-embedding-3-small

 Ask Genie

Model provider	llm/v1/completions	llm/v1/chat	llm/v1/embeddings
Anthropic	• claude-1 • claude-1.3-100k • claude-2 • claude-2.1 • claude-2.0 • claude-instant-1.2	• claude-3-5-sonnet-latest • claude-3-5-haiku-latest • claude-3-5-opus-latest • claude-3-5-sonnet-20241022 • claude-3-5-haiku-20241022 • claude-3-5-sonnet-20240620 • claude-3-haiku-20240307 • claude-3-opus-20240229 • claude-3-sonnet-20240229	
Cohere**	• command • command-light	• command-r7b-12-2024 • command-r-plus-08-2024 • command-r-08-2024	• embed-english-v2.0 • embed-multilingual-v2.0 • embed-english-light-v2.0  Ask Genie

Model provider	llm/v1/completions	llm/v1/chat	llm/v1/embeddings
		• command-r-plus • command-r • command • command-light-nightly • command-light • command-nightly	• embed-english-v3.0 • embed-english-light-v3.0 • embed-multilingual-v3.0 • embed-multilingual-light-v3.0
Model Serving	Databricks serving endpoint	Databricks serving endpoint	Databricks serving endpoint
Amazon Bedrock	Anthropic: • claude-instant-v1 • claude-v2 Cohere: • command-text-v14 • command-light-text-v14 AI21 Labs: • j2-grande-instruct • j2-jumbo-instruct • j2-mid • j2-mid-v1	Anthropic: • claude-3-5-sonnet-20241022-v2:0 • claude-3-5-haiku-20241022-v1:0 • claude-3-opus-20240229-v1:0 • claude-3-sonnet-20240229-v1:0	Amazon: • titan-embed-text-v2:0 • titan-embed-text-v1 • titan-embed-g1-text-02 Cohere: • embed-english-v3 • embed-multilingual-v3

Model provider	llm/v1/completions	llm/v1/chat	llm/v1/embeddings
	- j2-ultra - j2-ultra-v1	- claude-3-5-sonnet-20240620-v1:0 Cohere: - command-r-plus-v1:0 - command-r-v1:0 Amazon: - nova-lite-v1:0 - nova-micro-v1:0 - nova-pro-v1:0	
AI21 Labs†	- j2-mid - j2-light - j2-ultra		
Google Cloud Vertex AI	text-bison	- chat-bison - gemini-pro - gemini-1.0-pro - gemini-1.5-pro - gemini-1.5-flash - gemini-2.0-flash	- text-embedding-004 - text-embedding-005 - textembedding-gecko

** Model provider supports fine-tuned completion and chat models. To query a fine-tuned model, populate the `name` field of the `external_model` configuration with the name of your fine-tuned model.

† Model provider supports custom completion models.

Create foundation model serving endpoints

To query and use foundation models in your AI applications, you must first create a model serving endpoint. Model Serving uses a unified API and UI for creating and updating foundation model serving endpoints.

- To create an endpoint that serves fine-tuned variants of foundation models made available using Foundation Model APIs provisioned throughput, see [Create your provisioned throughput endpoint using the REST API](#).
- For creating serving endpoints that access foundation models made available using the External models offering, see [Create an external model serving endpoint](#).

Query foundation model serving endpoints

After you create your serving endpoint you are able to query your foundation model. Model Serving uses a unified OpenAI-compatible API and SDK for querying foundation models. This unified experience simplifies how you experiment with and customize foundation models for production across supported clouds and providers.

See [Use foundation models](#).

Last updated on **May 11, 2026**

Use foundation models

In this article, you learn which options are available to write query requests for foundation models and how to send them to your model serving endpoint. You can query foundation models that are hosted by Databricks and foundation models hosted outside of Databricks.

For traditional ML or Python models query requests, see [Query serving endpoints for custom models](#).

Model Serving supports [Foundation Models APIs](#) and [external models](#) for accessing foundation models. Model Serving uses a unified OpenAI-compatible API and SDK for querying them. This makes it possible to experiment with and customize foundation models for production across supported clouds and providers.

Query options

Model Serving provides the following options for sending query requests to endpoints that serve foundation models:

Method	Details
OpenAI client	Query a model hosted by a Model Serving endpoint using the OpenAI client. Specify the model serving endpoint name as the <code>model</code> input. Supported for chat, embeddings, and completions models made available by Foundation Model APIs or external models.
AI Functions	Invoke model inference directly from SQL using the <code>ai_query</code> SQL function. See Example: Query a foundation model



Ask Genie

Method	Details
Serving UI	Select Query endpoint from the Serving endpoint page. Insert JSON format model input data and click Send Request . If the model has an input example logged, use Show Example to load it.
REST API	Call and query the model using the REST API. See POST /serving-endpoints/{name}/invocations for details. For scoring requests to endpoints serving multiple models, see Query individual models behind an endpoint .
MLflow Deployments SDK	Use MLflow Deployments SDK's predict() function to query the model.
Databricks Python SDK	Databricks Python SDK is a layer on top of the REST API. It handles low-level details, such as authentication, making it easier to interact with the models.

Requirements

- A [model serving endpoint](#).
- A Databricks workspace in a supported region.
 - [Foundation Model APIs regions](#)
 - [External models regions](#)
- To send a scoring request through the OpenAI client, REST API or MLflow Deployment SDK, you must have a Databricks API token.

❗ IMPORTANT

As a security best practice for production scenarios, Databricks recommends that you use [machine-to-machine OAuth tokens](#) for authentication during production.

For testing and development, Databricks recommends using a personal access token belonging to [service principals](#) instead of workspace users. To create tokens for service principals, see [Manage tokens for a service principal](#).

Install packages

After you have selected a querying method, you must first install the appropriate package to your cluster.

OpenAI client

REST API

MLflow Deployments SDK

Databricks Python S

To use the OpenAI client, the `databricks-openai` package needs to be installed on your cluster. This package provides an OpenAI client with authorization automatically configured to query generative AI models. Run the following in your notebook or your local terminal:

```
pip install -U databricks-openai
```

The following is only required when installing the package on a Databricks Notebook

Python

```
dbutils.library.restartPython()
```

Foundation model types

The following table summarizes the supported foundation models based on task type.

ⓘ IMPORTANT

Meta-Llama-3.1-405B-Instruct will be retired,

- Starting February 15, 2026 for pay-per-token workloads.
- Starting May 15, 2026 for provisioned throughput workloads.

 Ask Genie

See [Retired models](#) for the recommended replacement model and guidance for how to migrate during deprecation.

Task type	Description	Supported models	When to use? Recommended use cases
General purpose	Models designed to understand and engage in natural, multi-turn conversations. They are fine-tuned on large datasets of human dialogue, which enables them to generate contextually relevant responses, track conversational history, and provide coherent, human-like interactions across various topics.	The following are supported Databricks-hosted foundation models: databricks-gpt-5-5-pro databricks-gpt-5-5 databricks-gpt-5-4 databricks-gpt-5-4-mini databricks-gpt-5-4-nano databricks-gpt-5-3-codex databricks-gpt-5-2-codex databricks-gpt-5-2 databricks-gpt-5-1 databricks-gpt-5-1-codex-max	Recommended for scenarios where natural, multi-turn dialogue and contextual understanding are needed: - Virtual assistants - Customer support bots - Interactive tutoring systems.

Task type	Description	Supported models	When to use? Recommended use cases
		- databricks-gpt-5-1-codex-mini - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro - databricks-gemini-3-pro - databricks-gemini-3-1-flash-lite - databricks-gemini-3-flash - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen35-122b-a10b - databricks-qwen3-next-	

Task type	Description	Supported models	When to use? Recommended use cases
		80b-a3b-instruct • databricks- claude- sonnet-4-6 • databricks- claude- sonnet-4-5 • databricks- claude- haiku-4-5 • databricks- gpt-oss-20b • databricks- gpt-oss-120b • databricks- gemma-3-12b • databricks- claude- sonnet-4 • databricks- claude-opus- 4-7 • databricks- claude-opus- 4-6 • databricks- claude-opus- 4-5 • databricks- claude-opus- 4-1	

Task type	Description	Supported models	When to use? Recommended use cases
		- databricks-llama-4-maverick - databricks-meta-llama-3-3-70b-instruct - databricks-meta-llama-3-1-405b-instruct - databricks-meta-llama-3-1-8b-instruct The following are supported external models: - OpenAI GPT and o series models - Anthropic Claude models - Google Gemini models	
Embeddings	Embedding models are machine learning systems	The following are supported	Recommended for applications where semantic  Ask Genie

Task type	Description	Supported models	When to use? Recommended use cases
	that transform complex data—such as text, images, or audio—into compact numerical vectors called embeddings. These vectors capture the essential features and relationships within the data, allowing for efficient comparison, clustering, and semantic search.	Databricks-hosted foundation model: databricks-qwen3-embedding-0-6b databricks-gte-large-en databricks-bge-large-en The following are supported external models: - OpenAI text embedding models - Cohere text embedding models - Google text embedding models	understanding, similarity comparison, and efficient retrieval or clustering of complex data are essential: - Semantic search - Retrieval augmented generation (RAG) - Topic clustering - Sentiment analysis and text analytics
Vision	Models designed to process, interpret, and analyze visual data—such as images and videos so machines can "see" and	The following are supported Databricks-hosted foundation models: databricks-gpt-5-5-pro	Recommended wherever automated, accurate, and scalable analysis of visual information is needed:  Ask Genie

Task type	Description	Supported models	When to use? Recommended use cases
	understand the visual world.	• databricks-gpt-5-5 • databricks-gpt-5-4 • databricks-gpt-5-4-mini • databricks-gpt-5-4-nano • databricks-gpt-5-2 • databricks-gpt-5-1 • databricks-gpt-5 • databricks-gpt-5-mini • databricks-gpt-5-nano • databricks-gemini-3-1-pro • databricks-gemini-3-pro • databricks-gemini-3-1-flash-lite • databricks-gemini-3-flash • databricks-gemini-2-5-	• Object detection and recognition • Image classification • Image segmentation • Document understanding

Task type	Description	Supported models	When to use? Recommended use cases
		pro - databricks-gemini-2-5-flash - databricks-gemma-3-12b - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-claude-haiku-4-5 - databricks-claude-sonnet-4 - databricks-claude-opus-4-7 - databricks-claude-opus-4-6 - databricks-claude-opus-4-5 - databricks-claude-opus-4-1 - databricks-llama-4-	

Task type	Description	Supported models	When to use? Recommended use cases
		maverick The following are supported external models: • OpenAI GPT and o series models with vision capabilities • Anthropic Claude models with vision capabilities • Google Gemini models with vision capabilities • Other external foundation models with vision capabilities that are OpenAI API compatible are also supported.	
Reasoning	Advanced AI systems designed	The following are supported	Recommended wherever automated,  Ask Genie

Task type	Description	Supported models	When to use? Recommended use cases
	to simulate human-like logical thinking. Reasoning models integrate techniques such as symbolic logic, probabilistic reasoning, and neural networks to analyze context, break down tasks, and explain their decision-making.	Databricks-hosted foundation model: - databricks-gpt-5-5-pro - databricks-gpt-5-5 - databricks-gpt-5-4 - databricks-gpt-5-4-mini - databricks-gpt-5-4-nano - databricks-gpt-5-2 - databricks-gpt-5-1 - databricks-gpt-5 - databricks-gpt-5-mini - databricks-gpt-5-nano - databricks-gemini-3-1-pro - databricks-gemini-3-pro - databricks-gemini-3-1-flash-lite	accurate, and scalable analysis of visual information is needed: - Code generation - Content creation and summarization - Agent orchestration

Task type	Description	Supported models	When to use? Recommended use cases
		- databricks-gemini-3-flash - databricks-gemini-2-5-pro - databricks-gemini-2-5-flash - databricks-qwen35-122b-a10b - databricks-claude-sonnet-4-6 - databricks-claude-sonnet-4-5 - databricks-gpt-oss-20b - databricks-gpt-oss-120b - databricks-claude-sonnet-4 - databricks-claude-opus-4-7 - databricks-claude-opus-4-6	

Task type	Description	Supported models	When to use? Recommended use cases
		<code>databricks- claude-opus- 4-5</code> <code>databricks- claude-opus- 4-1</code> The following are supported external models: - OpenAI models with reasoning capabilities - Anthropic Claude models with reasoning capabilities - Google Gemini models with reasoning capabilities	

Function calling

Databricks Function Calling is OpenAI-compatible and is only available during model serving as part of [Foundation Model APIs](#) and serving endpoints that serve [external models](#). For details, see [Function calling on Databricks](#).

Structured outputs

Structured outputs is OpenAI-compatible and is only available during model serving as part of [Foundation Model APIs](#). For details, see [Structured outputs on Databricks](#).

Prompt caching

Prompt caching is supported for Databricks-hosted Claude models as part of [Foundation Model APIs](#).

You can specify the `cache_control` parameter in your query requests to cache the following:

- Text content messages in the `messages.content` array.
- Thinking messages content in the `messages.content` array.
- Images content blocks in the `messages.content` array.
- Tool use, results and definitions in the `tools` array.

See [Foundation model REST API reference](#).

TextContent

ReasonContent

ImageContent

ToolCallContent

JSON

```
{
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "What's the date today?",
          "cache_control": { "type": "ephemeral" }
        }
      ]
    }
  ]
}
```

NOTE

The Databricks REST API is OpenAI-compatible and differs from the Anthropic API. These differences also impact response objects like the following:

- Output is returned in the `choices` field.
- Streaming chunk format. All chunks adhere to the same format where `choices` contains the response `delta` and usage is returned in every chunk.
- Stop reason is returned in the `finish_reason` field.
 - Anthropic uses: `end_turn`, `stop_sequence`, `max_tokens`, and `tool_use`
 - Respectively, Databricks uses: `stop`, `stop`, `length`, and `tool_calls`

Chat with supported LLMs using AI Playground

You can interact with supported large language models using the [AI Playground](#). The AI Playground is a chat-like environment where you can test, prompt, and compare LLMs from your Databricks workspace.

The screenshot shows the Databricks AI Playground interface. On the left is a sidebar with navigation options: New, Workspace, Recents, Catalog, Workflows, Compute, SQL (SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses), Data Engineering (Job Runs, Data Ingestion, Delta Live Tables), Machine Learning (Playground, Experiments, Features, Models, Serving), Marketplace, and Partner Connect. The main area is titled 'Playground' and includes a 'Preview' button and a 'Provide feedback' link. Below this, there's a dropdown for 'Meta Llama 3.1 70B Instruct', a 'Tools' dropdown, and an 'Export' button. The interface prompts the user to 'Try a sample question' and lists four examples: 'Function calling' (weather in San Francisco), 'Function calling' (distance between San Francisco and New York), 'Summarization', and 'Document Q&A'. At the bottom, there's a text input field with a placeholder 'Start typing ...' and a 'Send' button. The 'Ask Genie' logo is in the bottom right corner.

Additional resources

- [Monitor served models using Unity AI Gateway-enabled inference tables](#)
- [Deploy batch inference pipelines](#)
- [Databricks Foundation Model APIs](#)
- [External models in Model Serving](#)
- [Tutorial: Create external model endpoints to query OpenAI models](#)
- [Databricks-hosted foundation models available in Foundation Model APIs](#)
- [Foundation model REST API reference](#)

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 11, 2026**

Create foundation model serving endpoints

In this article, you learn how to create model serving endpoints that deploy and serve foundation models.

[Model Serving](#) supports the following models:

- [External models](#). These are foundation models that are hosted outside of Databricks. Endpoints that serve external models can be centrally governed and customers can establish rate limits and access control for them. Examples include foundation models like OpenAI's GPT-4 and Anthropic's Claude.
- State-of-the-art open foundation models made available by [Foundation Model APIs](#). These models are curated foundation model architectures that support optimized inference. Base models, like Meta-Llama-3.3-70B-Instruct and GTE-Large, are available for immediate use with **pay-per-token** pricing. Production workloads, using base or fine-tuned models, can be deployed with performance guarantees using **provisioned throughput**.

Model Serving provides the following options for model serving endpoint creation:

- The Serving UI
- REST API
- MLflow Deployments SDK

For creating endpoints that serve traditional ML or Python models, see [Create custom model serving endpoints](#).

Requirements

- A Databricks workspace in a supported region.
 - [Foundation Model APIs regions](#)
 - [External models regions](#)
- For creating endpoints using the MLflow Deployments SDK, you must install the MLflow Deployment client. To install it, run:

Python

```
import mlflow.deployments

client = mlflow.deployments.get_deploy_client("databricks")
```

Create a foundation model serving endpoint

You can create an endpoint that serves fine-tuned variants of foundation models made available using Foundation Model APIs **provisioned throughput**. See [Create your provisioned throughput endpoint using the REST API](#).

For foundation models that are made available using Foundation Model APIs **pay-per-token**, Databricks automatically provides specific endpoints to access the supported models in your Databricks workspace. To access them, select the **Serving** tab in the left sidebar of the workspace. The Foundation Model APIs are located at the top of the Endpoints list view.

For querying these endpoints, see [Use foundation models](#).

Create an external model serving endpoint

The following describes how to create an endpoint that queries a foundation model made available using Databricks external models.

1. In the **Name** field provide a name for your endpoint.
2. In the **Served entities** section
 - i. Click into the **Entity** field to open the **Select served entity** form.
 - ii. Select **Foundation models**.
 - iii. In the **Select a foundation model** field, select the model provider you want to use from those listed under **External model providers**. The form dynamically updates based on your model provider selection.
 - iv. Click **Confirm**.
 - v. Provide the configuration details for accessing the selected model provider. This is typically the secret that references the [personal access token](#) you want the endpoint to use to access this model.
 - vi. Select the task. Available tasks are chat, completion, and embeddings.
 - vii. Select the name of the external model you want to use. The list of models dynamically updates based on your task selection. See the [available external models](#).
3. Click **Create**. The **Serving endpoints** page appears with **Serving endpoint state** shown as Not Ready.

[Serving endpoints](#) >

Create serving endpoint

Experimenting with LLMs? Try [pay-per-token Foundation Model APIs](#)!

General

Name
Endpoint name cannot be changed after creation.

URL preview: <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/>

Served entities

Entity details×

Entity	Version	Traffic (%)
Select an entity	▼	100

+ Add served entity

AI Gateway [Preview](#) >

Monitor and secure endpoints. [Learn More](#) [Learn more about billing](#).

Route optimization

☐ Enable route optimization
Recommended for high throughput or low latency workloads. Route optimization cannot be changed after endpoint creation. [Learn more](#).

Tags >

Optional.

Update model serving endpoints

After enabling a model endpoint, you can set the compute configuration as desired. This configuration is particularly helpful if you need additional resources for your model. Workload size and compute configuration play a key role in what resources are allocated for serving your model.

Until the new configuration is ready, the old configuration keeps serving prediction traffic. While there is an update in progress, another update cannot be made. In the Serving UI, you can cancel an in progress configuration update by selecting **Cancel update** on the top right of the endpoint's details page. This functionality is only available in the Serving UI.

When an `external_model` is present in an endpoint configuration, the served entities list can only have one `served_entity` object. Existing endpoints with an `external_model` can not be updated to no longer have an `external_model`. If the

endpoint is created without an `external_model`, you cannot update it to add an `external_model`.

REST API MLflow Deployments SDK

To update your endpoint see the REST API [update configuration documentation](#) for request and response schema details.

Bash

```
{
  "name": "openai_endpoint",
  "served_entities":
  [
    {
      "name": "openai_chat",
      "external_model": {
        "name": "gpt-4",
        "provider": "openai",
        "task": "llm/v1/chat",
        "openai_config": {
          "openai_api_key": "{{secrets/my_scope/my_openai_api_key}}"
        }
      }
    }
  ]
}
```

Additional resources

- [AI Gateway-enabled inference tables](#)
- [Use foundation models.](#)
- [External models in Model Serving.](#)

Last updated on **May 7, 2026**

Applicable model terms

This document lists models in the Databricks Services that are subject to separate model terms, in addition to your underlying agreement with Databricks. These links are provided for your convenience. You are responsible for ensuring compliance with applicable model terms and any applicable use policies.

Models and terms

Following is a table of models with their terms, and other pertinent information.

Model	Terms and policies
OpenAI GPT-5.5 Pro	OpenAI high risk use case mitigation requirements Usage policy
OpenAI GPT-5.5	OpenAI high risk use case mitigation requirements Usage policy
OpenAI GPT-5.4	OpenAI high risk use case mitigation requirements Usage policy
OpenAI GPT-5.4 mini	OpenAI high risk use case mitigation requirements Usage policy
OpenAI GPT-5.4 nano	OpenAI high risk use case mitigation requirements Usage policy
 Ask Genie	

Model	Terms and policies
OpenAI GPT-5.3 Codex	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5.2 Codex	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5.2	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5.1	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5.1 Codex Max	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5.1 Codex Mini	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5 mini	• OpenAI high risk use case mitigation requirements • Usage policy
OpenAI GPT-5 nano	• OpenAI high risk use case mitigation requirements • Usage policy
Google Gemini 3.1 Pro Preview	• Databricks Terms of Use – Google Models • Acceptable use policy • Generative AI prohibited use policy
Google Gemini 3.1 Flash Lite	• Databricks Terms of Use – Google Models • Acceptable use policy  Ask Genie

Model	Terms and policies
	Generative AI prohibited use policy
Google Gemini 3 Pro Preview	- Databricks Terms of Use – Google Models - Acceptable use policy - Generative AI prohibited use policy
Google Gemini 3 Flash	- Databricks Terms of Use – Google Models - Acceptable use policy - Generative AI prohibited use policy
Google Gemini 2.5 Pro	- Databricks Terms of Use – Google Models - Acceptable use policy - Generative AI prohibited use policy
Google Gemini 2.5 Flash	- Databricks Terms of Use – Google Models - Acceptable use policy - Generative AI prohibited use policy
Anthropic Claude Sonnet 4.5	Usage policy
Anthropic Claude Haiku 4.5	Usage policy
Anthropic Claude Sonnet 4.6	Usage policy
Anthropic Claude Opus 4.7	Usage policy
Anthropic Claude Opus 4.6	Usage policy
 Ask Genie	

Model	Terms and policies
Anthropic Claude Opus 4.5	Usage policy
Anthropic Claude Opus 4.1	Usage policy
Google Gemma 3 12B	• Terms of use • Usage policy
Anthropic Claude Sonnet 4	Usage policy
Meta Llama 4 Maverick	• Llama 4 Community License, Copyright (c) Meta Platforms, Inc. All Rights Reserved. • Llama 4 Acceptable Use
Meta Llama 3.3 70B Instruct	• Llama 3.3 Community License, Copyright (c) Meta Platforms, Inc. All Rights Reserved. • Llama 3.3 Acceptable Use
Meta Llama 3.2 models	• Meta Llama 3.2 Community License, Copyright (c) Meta Platforms, Inc. All Rights Reserved. • Llama 3.2 Acceptable Use Policy
Meta Llama 3.1 8B Instruct	• Llama 3.1 Community License Agreement, Copyright (c) Meta Platforms, Inc. All Rights Reserved. • Llama 3.1 Acceptable Use Policy
Meta Llama 3.1 405B Instruct	• Llama 3.1 Community License Agreement, Copyright (c) Meta Platforms, Inc. All Rights Reserved. • Llama 3.1 Acceptable Use Policy

Additional resources

- Databricks-hosted foundation models available in Foundation Model APIs
- Supported foundation models on Model Serving
- Provisioned throughput Foundation Model APIs

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 7, 2026**

OpenAI high risk use case mitigation requirements

In this page, learn about the mitigation requirements for OpenAI high risk use cases. You are responsible for implementing these requirements when using OpenAI models on Databricks

Mitigation requirements

End users must implement the following requirements when using OpenAI for the following high-risk use cases:

High-risk use case	Description	Mitigation
Applications involving chat or conversations	Applications that enable users to interact with a conversational agent	Verify that such applications are grounded or topical meaning that users interacting with the application do not have unrestricted access to query the model with general inputs that generate unrestricted outputs.
Applications accessible to users outside your organization		Authenticate or monitor such users through one of the following mechanisms: • Two factor or multi-factor authentication.

High-risk use case	Description	Mitigation
		• Logging of individual end user IDs for visibility and remediation. • Logging of individual IP addresses for visibility and remediation.
Applications involving code generation or transformation scenarios		• Conduct human review of any code before it is used in production. • Limit user-based risk by either restricting code generation to internal users or implementing client-side monitoring for misuse.
Applications enabling image inputs		You are required to ensure that such inputs are restricted to low risk and topical images.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Custom models overview

This article describes support for **custom models** using [Model Serving](#). It provides details about supported model logging options and compute types, how to package model dependencies for serving, and expectations for endpoint creation and scaling.

What are custom models?

Model Serving can deploy any Python model or [custom code](#) as a production-grade API using CPU or GPU compute resources. Databricks refers to such models as **custom models**. These ML models can be trained using standard ML libraries like scikit-learn, XGBoost, PyTorch, and HuggingFace transformers and can include any Python code.

To deploy a **custom model**,

1. Log the model or code in the MLflow format, using either native [MLflow built-in flavors](#) or [pyfunc](#).
2. After the model is logged, register it in the Unity Catalog (recommended) or the workspace registry.
3. From here, you can create a model serving endpoint to deploy and query your model.
 - i. See [Create custom model serving endpoints](#)
 - ii. See [Query serving endpoints for custom models](#).

For a complete tutorial on how to serve custom models on Databricks, see [Model serving tutorial](#).

Databricks also supports serving foundation models for generative AI applications, see [Foundation Model APIs](#) and [External models](#) for supported models and compute offerings.

Log ML models

There are different methods to log your ML model for model serving. The following list summarizes the supported methods and examples.

- **Autologging** This method is automatically enabled when using Databricks Runtime for ML.

Python

```
import mlflow
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import load_iris

iris = load_iris()
model = RandomForestRegressor()
model.fit(iris.data, iris.target)
```

- **Log using MLflow's built-in flavors.** You can use this method if you want to manually log the model for more detailed control.

Python

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

iris = load_iris()
model = RandomForestClassifier()
model.fit(iris.data, iris.target)

with mlflow.start_run():
    mlflow.sklearn.log_model(model, "random_forest_classifier")
```

- **Custom logging with `pyfunc`.** You can use this method for deploying arbitrary python code models or deploying additional code alongside your model.

Python

```
import mlflow
import mlflow.pyfunc
```

 Ask Genie

```
class Model(mlflow.pyfunc.PythonModel):
    def predict(self, context, model_input):
        return model_input * 2

with mlflow.start_run():
    mlflow.pyfunc.log_model("custom_model", python_model=Model())
```

Signature and input examples

Adding a signature and input example to MLflow is recommended. Signatures are necessary for logging models to the Unity Catalog.

The following is a signature example:

Python

```
from mlflow.models.signature import infer_signature

signature = infer_signature(training_data, model.predict(training_data))
mlflow.sklearn.log_model(model, "model", signature=signature)
```

The following is an input example:

Python

```
input_example = {"feature1": 0.5, "feature2": 3}
mlflow.sklearn.log_model(model, "model", input_example=input_example)
```

Compute type

Model Serving provides a variety of CPU and GPU options for deploying your model. When deploying with a GPU, you must make sure that your code is set up so that predictions are run on the GPU, using the methods provided by your framework. MLflow does this automatically for models logged with the PyTorch or Transformers flavors.

The `CPU_MEDIUM` and `CPU_LARGE` workload types are in [Beta](#) and let you trade concurrency for more memory per worker on the same CPU hardware. Use them when your model needs more memory than standard `CPU` provides.

Workload type	GPU instance	Memory
<code>CPU</code>		4GB per concurrency
<code>CPU_MEDIUM</code> (Beta)		8GB per concurrency
<code>CPU_LARGE</code> (Beta)		16GB per concurrency
<code>GPU_SMALL</code>	1xT4	16GB per concurrency
<code>GPU_MEDIUM</code>	1xA10G	24GB per concurrency
<code>MULTIGPU_MEDIUM</code>	4xA10G	96GB per concurrency
<code>GPU_MEDIUM_8</code>	8xA10G	192GB per concurrency

Deployment container and dependencies

During deployment, a production-grade container is built and deployed as the endpoint. This container includes libraries automatically captured or specified in the MLflow model. The base image may include some system-level dependencies, but application-level dependencies must be explicitly specified in your MLflow model.

If not all required dependencies are included in the model, you might encounter dependency errors during deployment. When running into model deployment issues, Databricks recommends you test the model locally.

Package and code dependencies

Custom or private libraries can be added to your deployment. See [Use custom Python libraries with Model Serving](#).

For MLflow native flavor models, the necessary package dependencies are automatically captured.

For custom `pyfunc` models, dependencies can be explicitly added. For detailed information about logging requirements and best practices, see the [MLflow Models documentation](#) and [MLflow Python API reference](#).

You can add package dependencies using:

- The `pip_requirements` parameter:

Python

```
mlflow.sklearn.log_model(model, "sklearn-model", pip_requirements =
["scikit-learn", "numpy"])
```

- The `conda_env` parameter:

Python

```
conda_env = {
    'channels': ['defaults'],
    'dependencies': [
        'python=3.7.0',
        'scikit-learn=0.21.3'
    ],
    'name': 'mlflow-env'
}

mlflow.sklearn.log_model(model, "sklearn-model", conda_env =
conda_env)
```

- To include additional requirements beyond what is automatically captured, use `extra_pip_requirements`.

Python

 Ask Genie

```
mlflow.sklearn.log_model(model, "sklearn-model",  
extra_pip_requirements = ["sklearn_req"])
```

If you have code dependencies, these can be specified using `code_path`.

Python

```
mlflow.sklearn.log_model(model, "sklearn-model", code_path=  
["path/to/helper_functions.py"],)
```

For information about validating and updating dependencies before deployment, see [Pre-deployment validation for Model Serving](#).

Expectations and limitations

NOTE

The information in this section does not apply to endpoints that serve foundation models or external models.

The following sections describe known expectations and limitations for serving custom models using Model Serving.

Endpoint creation and update expectations

- **Deployment time:** Deploying a newly registered model version involves packaging the model and its model environment and provisioning the model endpoint itself. This process can take approximately 10 minutes, but may take longer depending on model complexity, size, and dependencies.
- **Zero-downtime updates:** Databricks performs a zero-downtime update of endpoints by keeping the existing endpoint configuration up until the new one becomes ready. Doing so reduces risk of interruption for endpoints that are in use. During this update process, you are billed for both the old and new endpoint configurations until the transition is complete.
- **Request timeout:** If model computation takes longer than 597 seconds, requests will time out.

❗ IMPORTANT

Databricks performs occasional zero-downtime system updates and maintenance on existing Model Serving endpoints. During maintenance, Databricks reloads models. If a model fails to reload, the endpoint update is marked as failed and the existing endpoint configuration continues to serve requests. Make sure your customized models are robust and are able to reload at any time.

Endpoint scaling expectations

Serving endpoints automatically scale based on traffic and the capacity of provisioned concurrency units.

- **Provisioned concurrency:** The maximum number of parallel requests the system can handle. Estimate the required concurrency using the formula: $\text{provisioned concurrency} = \text{queries per second (QPS)} * \text{model execution time (s)}$. To validate your concurrency configuration, see [Load testing for serving endpoints](#).
- **Scaling behavior:** Endpoints scale up almost immediately with increased traffic and scale down every five minutes to match reduced traffic.
- **Scale to zero:** Scale to zero is an optional feature for endpoints that allows them to scale down to zero after 30 minutes of inactivity. The first request after scaling to zero experiences a "cold start," leading to higher latency. Scaling up from zero usually takes 10–20 seconds, but can sometimes take minutes. There is no SLA on scale from zero latency.
- **Route optimization:** For high QPS and low latency use cases, [route optimization](#) is the optimal and recommended option to improve performance.
- **Serverless optimized deployments:** For faster endpoint deployment speed, use [serverless optimized deployments](#).

⚠ WARNING

Scale to zero should not be used for production workloads that require consistent uptime or guaranteed response times. For latency-sensitive applications or endpoints requiring continuous availability, disable scale to zero.

GPU workload limitations

The following are limitations for serving endpoints with GPU workloads:

- Container image creation for GPU serving takes longer than image creation for CPU serving due to model size and increased installation requirements for models served on GPU.
- When deploying very large models, the deployment process might timeout if the container build and model deployment exceed a 60-minute duration, or the container build might fail with "No space left on device" error due to storage limitations. For large language models, use [Foundation Model APIs](#) instead.
- Autoscaling for GPU serving takes longer than for CPU serving.
- GPU capacity is not guaranteed when scaling to zero. GPU endpoints might expect extra high latency for the first request after scaling to zero.

Anaconda licensing notice for legacy models

NOTE

This section applies only to models logged with MLflow v1.17 or earlier (Databricks Runtime 8.3 ML or earlier). If you are using a newer version, you can skip this section.

The following notice is for customers relying on Anaconda with legacy models.

IMPORTANT

Anaconda Inc. updated their [terms of service](#) for anaconda.org channels. Based on the new terms of service you may require a commercial license if you rely on Anaconda's packaging and distribution. See [Anaconda Commercial Edition FAQ](#) for more information. Your use of any Anaconda channels is governed by their terms of service.

MLflow models logged before [v1.18](#) (Databricks Runtime 8.3 ML or earlier) were by default logged with the `conda defaults` channel  Ask Genie (<https://repo.anaconda.com/pkgs/>) as a dependency. Because of this license

change, Databricks has stopped the use of the `defaults` channel for models logged using MLflow v1.18 and above. The default channel logged is now `conda-forge`, which points at the community managed <https://conda-forge.org/>.

If you logged a model before MLflow v1.18 without excluding the `defaults` channel from the conda environment for the model, that model may have a dependency on the `defaults` channel that you may not have intended. To manually confirm whether a model has this dependency, you can examine `channel` value in the `conda.yaml` file that is packaged with the logged model. For example, a model's `conda.yaml` with a `defaults` channel dependency may look like this:

YAML

```
channels:
- defaults
dependencies:
- python=3.8.8
- pip
- pip:
  - mlflow
  - scikit-learn==0.23.2
  - cloudpickle==1.6.0
  name: mlflow-env
```

Because Databricks can not determine whether your use of the Anaconda repository to interact with your models is permitted under your relationship with Anaconda, Databricks is not forcing its customers to make any changes. If your use of the Anaconda.com repo through the use of Databricks is permitted under Anaconda's terms, you do not need to take any action.

If you would like to change the channel used in a model's environment, you can re-register the model to the model registry with a new `conda.yaml`. You can do this by specifying the channel in the `conda_env` parameter of `log_model()`.

For more information on the `log_model()` API, see the MLflow documentation for the model flavor you are working with, for example, [log_model for scikit-learn](#).

For more information on `conda.yaml` files, see the [MLflow documentation](#).

Additional resources

- [Create custom model serving endpoints](#)
- [Query serving endpoints for custom models](#)
- [Debugging guide for Model Serving](#)
- [Use custom Python libraries with Model Serving](#)
- [Package custom artifacts for Model Serving](#)
- [Deploy Python code with Model Serving](#)
- [Route optimization on serving endpoints](#)
- [Serverless optimized deployments for model serving endpoints](#)
- [Configure access to resources from model serving endpoints](#)
- [Add an instance profile to a model serving endpoint](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 13, 2026**

Query serving endpoints for custom models

In this article, learn how to format scoring requests for your served model, and how to send those requests to the model serving endpoint. The guidance is relevant to serving [custom models](#), which Databricks defines as traditional ML models or customized Python models packaged in the MLflow format. Register the models in Unity Catalog or in the workspace model registry. Examples include scikit-learn, XGBoost, PyTorch, and Hugging Face transformer models. See [Deploy models using Model Serving](#) for more information about this functionality and supported model categories.

For query requests for generative AI and LLM workloads, see [Use foundation models](#).

Requirements

- A [model serving endpoint](#).
- For the MLflow Deployment SDK, MLflow 2.9 or above is required.
- [Scoring request in an accepted format](#).
- To send a scoring request through the REST API or MLflow Deployment SDK, you must have a Databricks API token.

❗ IMPORTANT

As a security best practice for production scenarios, Databricks recommends that you use [machine-to-machine OAuth tokens](#) for authentication during production.

For testing and development, Databricks recommends using a personal token belonging to [service principals](#) instead of workspace users. To c

 Ask Genie

tokens for service principals, see [Manage tokens for a service principal](#).

Querying methods and examples

Model Serving provides the following options for sending scoring requests to served models:

Method	Details
Serving UI	Select Query endpoint from the Serving endpoint page in your Databricks workspace. Insert JSON format model input data and click Send Request . If the model has an input example logged, use Show Example to load it.
SQL function	Invoke model inference directly from SQL using the <code>ai_query</code> SQL function. See ai_query function .
REST API	Call and query the model using the REST API. See POST /serving-endpoints/{name}/invocations for details. For scoring requests to endpoints serving multiple models, see Query individual models behind an endpoint .
MLflow Deployments SDK	Use MLflow Deployments SDK's <code>predict()</code> function to query the model.

Pandas DataFrame scoring example

The following example assumes a `ENDPOINT_INVOCATION_URL` like `https://<databricks-instance>/serving-endpoints/<endpoint-name>/invocations`, where `<databricks-instance>` is the [name of your Databricks instance](#), and a [Databricks REST API token](#) called `DATABRICKS_API_TOKEN`.

See [Supported scoring formats](#).

Score a model accepting dataframe split input format.

Bash

```
curl -X POST -u token:$DATABRICKS_API_TOKEN $ENDPOINT_INVOCATION_URL \
-H 'Content-Type: application/json' \
-d '{"dataframe_split": [{
  "columns": ["sepal length (cm)", "sepal width (cm)", "petal length
(cm)", "petal width (cm)"],
  "data": [[5.1, 3.5, 1.4, 0.2], [4.9, 3.0, 1.4, 0.2]]
}]}'
```

Score a model accepting tensor inputs. Tensor inputs should be formatted as described in [TensorFlow Serving's API documentation](#).

Bash

```
curl -X POST -u token:$DATABRICKS_API_TOKEN $ENDPOINT_INVOCATION_URL \
-H 'Content-Type: application/json' \
-d '{"inputs": [[5.1, 3.5, 1.4, 0.2]]}'
```

Tensor input example

The following example scores a model accepting tensor inputs. Tensor inputs should be formatted as described in [TensorFlow Serving's API docs](#). This example assumes a `ENDPOINT_INVOCATION_URL` like `https://<databricks-instance>/serving-endpoints/<endpoint-name>/invocations`, where `<databricks-instance>` is the [name of your Databricks instance](#), and a [Databricks REST API token](#) called `DATABRICKS_API_TOKEN`.

Bash

```
curl -X POST -u token:$DATABRICKS_API_TOKEN $ENDPOINT_INVOCATION_URL \
-H 'Content-Type: application/json' \
-d '{"inputs": [[5.1, 3.5, 1.4, 0.2]]}'
```

Supported scoring formats

For custom models, Model Serving supports scoring requests in Pandas DataFrame or Tensor input.

Pandas DataFrame

Requests should be sent by constructing a JSON-serialized Pandas DataFrame with one of the supported keys and a JSON object corresponding to the input format.

- (Recommended) `dataframe_split` format is a JSON-serialized Pandas DataFrame in the `split` orientation.

JSON

```
{
  "dataframe_split": {
    "index": [0, 1],
    "columns": ["sepal length (cm)", "sepal width (cm)", "petal
length (cm)", "petal width (cm)"],
    "data": [
      [5.1, 3.5, 1.4, 0.2],
      [4.9, 3.0, 1.4, 0.2]
    ]
  }
}
```

- `dataframe_records` is JSON-serialized Pandas DataFrame in the `records` orientation.

NOTE

This format does not guarantee the preservation of column ordering, and the `split` format is preferred over the `records` format.

JSON

```
{
  "dataframe_records": [
    {
```

 Ask Genie

```

    "sepal length (cm)": 5.1,
    "sepal width (cm)": 3.5,
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2
  },
  {
    "sepal length (cm)": 4.9,
    "sepal width (cm)": 3,
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2
  },
  {
    "sepal length (cm)": 4.7,
    "sepal width (cm)": 3.2,
    "petal length (cm)": 1.3,
    "petal width (cm)": 0.2
  }
]
}

```

The response from the endpoint contains the output from your model, serialized with JSON, wrapped in a `predictions` key.

JSON

```

{
  "predictions": [0, 1, 1, 1, 0]
}

```

Tensor input

When your model expects tensors, like a TensorFlow or Pytorch model, there are two supported format options for sending requests: `instances` and `inputs`.

If you have multiple named tensors per row, then you have to have one of each tensor for every row.

- `instances` is a tensors-based format that accepts tensors in row format. Use this format if all the input tensors have the same 0-th dimension. Conceptually, each tensor in the instances list could be joined with the other tensors of the same name in the rest of the list to construct the full input tensor for the model, which would only be possible if all of the tensors have the same 0-th dimension.

JSON

```
{ "instances": [1, 2, 3] }
```

The following example shows how to specify multiple named tensors.

JSON

```
{
  "instances": [
    {
      "t1": "a",
      "t2": [1, 2, 3, 4, 5],
      "t3": [
        [1, 2],
        [3, 4],
        [5, 6]
      ]
    },
    {
      "t1": "b",
      "t2": [6, 7, 8, 9, 10],
      "t3": [
        [7, 8],
        [9, 10],
        [11, 12]
      ]
    }
  ]
}
```

- `inputs` send queries with tensors in columnar format. This request is different because there are actually a different number of tensor instances of `t2` (3) than `t1` and `t3`, so it is not possible to represent this input in the `instances` format.

JSON

```
{
  "inputs": {
    "t1": ["a", "b"],
    "t2": [
      [1, 2, 3, 4, 5],
      [6, 7, 8, 9, 10]
    ]
  }
}
```

```
],  
  "t3": [  
    [  
      [1, 2],  
      [3, 4],  
      [5, 6]  
    ],  
    [  
      [7, 8],  
      [9, 10],  
      [11, 12]  
    ]  
  ]  
}
```

The response from the endpoint is in the following format.

JSON

```
{  
  "predictions": [0, 1, 1, 1, 0]  
}
```

Notebook example

See the following notebook for an example of how to test your Model Serving endpoint with a Python model:

Test Model Serving endpoint notebook

[Open notebook in new tab](#)

 [Copy link for import](#)

Test Serverless endpoint by querying your model

The notebook loads an input example that was logged with the registered model, `ElasticNetDiabetes`, and queries the model to test the serverless endpoint.

Prerequisites

- Logged and registered Python-based model in the Model Registry
- Model Serving endpoint (AWS ([https://docs.databricks.com/machine-](https://docs.databricks.com/machine-learning/model-serving/index.html)

Expand notebook ▾

el-serving/index.html)|Azure

([https://docs.microsoft.com/azure/databricks/machine-learning/model-](https://docs.microsoft.com/azure/databricks/machine-learning/model-serving/index.html)

Additional resources

- [Monitor served models using Unity AI Gateway-enabled inference tables.](#)
- [Use foundation models.](#)
- [Debugging guide for Model Serving.](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 7, 2026**

Serverless optimized deployments for model serving endpoints

This article describes how to use serverless optimized deployments on your [model serving](#) endpoints. Serverless optimized deployments dramatically lower deployment times and keep the model serving environment the same as the model training environment.

What are serverless optimized deployments?

Serverless optimized deployments take advantage of packaging and staging model artifacts in serverless notebook environments during model registration, resulting in accelerated endpoint deployment and consistent environments between training and serving.

This differs from non-serverless optimized deployments, where model artifacts and environments are packaged into containers at deployment time. In such cases, the serving environment may not match the one used during model training.

Requirements

Serverless optimized endpoints have the same requirements as model serving endpoint (see [Requirements](#)). In addition:

- The model must be a custom model (not [FMAPI](#))

- The model must be logged and registered in a [Serverless Notebook](#) using [version 3](#) or 4
- The model must be logged and registered with `mlflow>=3.1`
- The model must be registered in UC and served with CPU
- The model's max environment size is 1GB

Using serverless optimized deployments

When logging and registering a model, use a Serverless Notebook with client 3 or 4 and `mlflow>=3.1`.

To adjust the client version of the serverless environment, see [Configure the serverless environment](#).

Then, when registering a model, set the `env_pack` parameter with the desired values.

Python

```
import mlflow
from mlflow.utils.env_pack import EnvPackConfig

mlflow.register_model(
    model_info.model_uri,
    model_name,
    env_pack=EnvPackConfig(name="databricks_model_serving")
)
```

Adding in the `env_pack` parameter will make the function pack and stage the model artifacts and serverless notebook environment during model registration to prepare it for usage during deployment. This may take additional time compared to registering the model without `env_pack`.

`EnvPackConfig` has a parameter `install_dependencies` (`True` by default) that determines whether the model's dependencies are installed in the current environment to confirm the environment is valid. If you'd like to skip that step, set the value to `False`.

 Ask Genie

NOTE

Endpoints in workspaces without internet access or endpoints with dependencies on custom libraries may fail if `install_dependencies` is set to `True`. In these cases, set `install_dependencies` to `False`.

You can also substitute `EnvPackConfig(...)` with `"databricks_model_serving"` as a shorthand. This is equivalent to `EnvPackConfig(name="databricks_model_serving", install_dependencies = True)`.

After registering the model is finished, you can [deploy the model in model serving](#). Notice that the deployment time is reduced and the event logs no longer indicate container build.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 11, 2026**

Pre-deployment validation for Model Serving

The guidance in this article can help you catch issues with your model before waiting for the endpoint deployment process. Databricks recommends going through these validation steps to ensure a better development experience when using model serving.

Test predictions before deployment

Before deploying your model to the serving endpoint, test offline predictions with a virtual environment using `mlflow.models.predict` and input examples. MLflow provides validation APIs that simulate the deployment environment and allow testing of modified dependencies.

There are two pre-deployment validation options: the [MLflow Python API](#) and the [MLflow CLI](#). See [MLflow documentation for testing predictions](#) for more detailed guidance.

You can specify the following parameters:

- The `model_uri` of the model that is deployed to model serving.
- One of the following:
 - The `input_data` in the expected format for the `mlflow.pyfunc.PyFuncModel.predict()` call of the model.
 - The `input_path` that defines a file containing input data that will be loaded and used for the call to `predict`.
- The `content_type` in `csv` or `json` format.
- An optional `output_path` to write the predictions to a file. If you omit this parameter, the predictions are printed to `stdout`.

- An environment manager, `env_manager`, that is used to build the environment for serving:
 - The default is `virtualenv`. Recommended for serving validation.
 - `local` is available, but potentially error prone for serving validation. Generally used only for rapid debugging.
- Whether to install the current version of MLflow that is in your environment with the virtual environment using `install_mlflow`. This setting defaults to `False`.
- Whether to update and test different versions of package dependencies for troubleshooting or debugging. You can specify this as a list of string dependency overrides or additions using the override argument, `pip_requirements_override`.

For example:

Python

```
import mlflow

run_id = "...
model_uri = f"runs://{run_id}/model"

mlflow.models.predict(
    model_uri=model_uri,
    input_data={"col1": 34.2, "col2": 11.2, "col3": "green"},
    content_type="json",
    env_manager="virtualenv",
    install_mlflow=False,
    pip_requirements_override=["pillow==10.3.0", "scipy==1.13.0"],
)
```

Update model dependencies

If there are any issues with the dependencies specified with a logged model, you can update the requirements by using the [MLflow CLI](#) or

`mlflow.models.model.update_model_requirements()` in the MLflow Python API without having to log another model.

The following example shows how to update the `pip_requirements.txt` of a logged model in-place.

You can update existing definitions with specified package versions or add non-existent requirements to the `pip_requirements.txt` file. This file is within the MLflow model artifact at the specified `model_uri` location.

Python

```
from mlflow.models.model import update_model_requirements

update_model_requirements(
    model_uri=model_uri,
    operation="add",
    requirement_list=["pillow==10.2.0", "scipy==1.12.0"],
)
```

Validate the model input before deployment

Model serving endpoints expect a special format of JSON input. You can validate that your model input works on a serving endpoint before deployment using `validate_serving_input` in MLflow.

The following is an example of the auto-generated code in the run's artifacts tab if your model is logged with a valid input example.

Python

```
from mlflow.models import validate_serving_input

model_uri = 'runs:/<run_id>/<artifact_path>'

serving_payload = """{
    "messages": [
        {
            "content": "How many product categories are there?",
            "role": "user"
        }
    ]
}
"""
```

```
# Validate the serving payload works on the model
validate_serving_input(model_uri, serving_payload)
```

You can also test any input examples against the logged model by using the `convert_input_example_to_serving_input` API to generate a valid JSON serving input.

Python

```
from mlflow.models import validate_serving_input
from mlflow.models import convert_input_example_to_serving_input

model_uri = 'runs:/<run_id>/<artifact_path>'

# Define INPUT_EXAMPLE with your own input example to the model
# A valid input example is a data instance suitable for pyfunc prediction

serving_payload = convert_input_example_to_serving_input(INPUT_EXAMPLE)

# Validate the serving payload works on the model
validate_serving_input(model_uri, serving_payload)
```

Manually test serving the model

You can manually test the serving behavior of the model using the following steps:

1. Open a notebook and attach to an All-Purpose cluster that uses a Databricks Runtime version, not Databricks Runtime for Machine Learning.
2. Load the model using MLflow and try debugging from there.

You can also load the model locally on your PC and debug from there. Load your model locally using the following:

Python

```
import os
import mlflow

os.environ["MLFLOW_TRACKING_URI"] = "databricks://PROFILE"

ARTIFACT_URI = "model_uri"
if '.' in ARTIFACT_URI:
```

 Ask Genie


```
mlflow.set_registry_uri('databricks-uc')
local_path = mlflow.artifacts.download_artifacts(ARTIFACT_URI)
print(local_path)

conda env create -f local_path/artifact_path/conda.yaml
conda activate mlflow-env

mlflow.pyfunc.load_model(local_path/artifact_path)
```

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 13, 2026**

Debugging guide for Model Serving

This article demonstrates debugging steps for common issues that users might encounter when working with model serving endpoints. Common issues could include errors users encounter when the endpoint fails to initialize or start, build failures related to the container, or problems during the operation or running of the model on the endpoint.

💡 VALIDATE BEFORE DEBUGGING

Having deployment issues? Start with [pre-deployment validation](#) to catch common problems before they occur.

Debug your container build

Databricks recommends reviewing logs for debugging and troubleshooting errors in your model serving workloads. See [Monitor model quality and endpoint health](#) for information about logs and how to view them.

The event logs (click the **Events** tab) in the workspace UI contain information about the progress of a container build. A successful container build is highlighted by a `SERVED_ENTITY_CONTAINER_EVENT` event type along with a message `Container image creation finished successfully`. If you do not see any build event or message after an hour of creating the endpoint, reach out to Databricks support for assistance.

If your build is successful, but you encounter other errors see [Debug after container build succeeds](#). If your build fails, see [Debug after container build failure](#).

Debug after container build succeeds

Even if the container builds successfully, there might be issues when you run the model or during the operation of the endpoint itself. The following subsections detail common issues and how to troubleshoot them.

NOTE

If your model code returns `MLflowException` errors, expect the response code to be mapped to a `4xx` response. Databricks considers these model code errors to be customer-caused errors, since they can be resolved based on the resulting error message. `5xx` error codes are reserved to communicate errors where Databricks is at fault.

Missing dependency

You might get an error like `An error occurred while loading the model. No module named <module-name>.`, which might indicate that a dependency is missing from the container. Verify that you properly denoted all the dependencies that should be included in the build of the container. Pay special attention to custom libraries and ensure that the `.whl` files are included as artifacts.

Model fails or times out when requests are sent to the endpoint

You might receive an error like `Encountered an unexpected error while evaluating the model.` Verify that the input is compatible with the model for inference. when `predict()` is called on your model.

This error can indicate a code issue in the `predict()` function. Databricks recommends that you load the model from MLflow in a notebook and call it. Doing so highlights the issues in the `predict()` function, and you can see where the failure is happening within the method.

Root cause analysis of failed requests

If a request to an endpoint fails, you can perform root cause analysis by using inference tables. If enabled, inference tables automatically log all requests and responses to your endpoint in a Unity Catalog table for you to query.

- See [Monitor served models using Unity AI Gateway-enabled inference tables](#).

To query inference tables:

1. In your workspace, go to the **Serving** tab and select your endpoint name.
2. In the **Inference tables** section, find the inference table's fully-qualified name. For example, `my-catalog.my-schema.my-table`.
3. Run the following in a Databricks notebook:

Python

```
%sql
SELECT * FROM my-catalog.my-schema.my-table
```

4. View and filter on columns such as `request`, `response`, `request_time` and `status_code` to understand the requests and narrow down results.

Python

```
%sql
SELECT * FROM my-catalog.my-schema.my-table
WHERE status_code != 200
```

5. If you enabled agent tracing for AI agents, see the **Response** column to view detailed traces. See [Enable inference tables for AI agents](#).

Workspace exceeds provisioned concurrency

You might receive a `Workspace exceeded provisioned concurrency quota` error. This indicates that you have reached your workspace quota for provisioned concurrency. See [Model Serving limits and regions](#) for more information about concurrency limits.

You can free up this quota by [deleting](#) or [stopping](#) unused endpoints.

This limit can be increased depending on region availability. Reach out to your  Ask Genie Databricks account team and provide your workspace ID to request a concurrency

increase.

Workspace exceeds parallel requests limit

You might receive the following 429 error: `Exceeded max number of parallel requests`. Please contact your Databricks representative to increase the `limit`. This limit indicates that you have reached the workspace limit on the maximum number of requests that can be sent in parallel. See [Model Serving limits and regions](#) for more information about this limit.

Databricks recommends moving to [route optimized endpoints](#), where this limit has been removed. If you cannot move to route optimized endpoints, you can either reduce the number of clients sending inference requests or contact your Databricks representative for a quota increase.

Too many concurrent requests

You might receive the following 429 error: `Too many concurrent requests. Consider increasing the provisioned concurrency of the served entity`. This error indicates that your endpoint's current provisioned concurrency cannot handle the incoming traffic volume. If you have enabled autoscaling for your endpoint, the system will automatically provision additional concurrency up to the endpoint's configured limit to handle the increased load. If autoscaling is not enabled, consider manually increasing the provisioned concurrency or enabling autoscaling to handle traffic spikes.

Debug after container build failure

This section details issues that might occur when your build fails.

`OSError: [Errno 28] No space left on device`

The `No space left` error can be due to too many large artifacts being logged alongside the model unnecessarily. Check in MLflow that extraneous artifacts are not logged alongside the model and try to redeploy the slimmed down package.

Build failure due to lack of GPU availability

Due to restrictions in GPU supply and availability, your GPU build may fail with this error: `Build could not start due to an internal error – please contact your Databricks representative.`

Reach out to your Databricks account team to help resolve. Depending on region availability, the team can provision more GPU resources.

Installed library package versions

Databricks recommends that you define all important libraries as model dependencies to ensure consistent and reproducible model behavior across environments. In the build logs you can confirm the package versions that are installed correctly.

- For MLflow versions, if you do not have a version specified, Model Serving uses the latest version.
- For custom GPU serving, Model Serving installs the recommended versions of `cuda` and `cuDNN` according to public PyTorch and Tensorflow documentation.

Log models that require `flash-attn`

If you are logging a model that requires `flash-attn`, Databricks recommends using a custom wheel version of `flash-attn`. Otherwise, build errors such as `ModuleNotFoundError: No module named 'torch'` can result.

To use a custom wheel version of `flash-attn`, specify all pip requirements as a list and pass it as a parameter into your `mlflow.transformers.log_model` function. You must also specify the `pytorch`, `torch`, and `torchvision` versions that are compatible with the CUDA version specified in your `flash_attn` wheel.

For example, Databricks recommends using the following versions and wheels for CUDA 11.8:

- [Pytorch](#)
- Torch 2.0.1+cu118
- Torchvision 0.15.2+cu118

- Flash-Attn

Python

```
logged_model=mlflow.transformers.log_model(  
transformers_model=test_pipeline,  
    artifact_path="artifact_path",  
    pip_requirements=["--extra-index-url  
https://download.pytorch.org/whl/cu118", "mlflow==2.13.1",  
"setuptools<70.0.0", "torch==2.0.1+cu118", "accelerate==0.31.0",  
"astunparse==1.6.3", "bcrypt==3.2.0", "boto3==1.34.39",  
"configparser==5.2.0", "defusedxml==0.7.1", "dill==0.3.6", "google-cloud-  
storage==2.10.0", "ipython==8.15.0", "lz4==4.3.2", "nvidia-ml-  
py==12.555.43", "optree==0.12.1", "pandas==1.5.3", "pyopenssl==23.2.0",  
"pytesseract==0.3.10", "scikit-learn==1.3.0", "sentencepiece==0.1.99",  
"torchvision==0.15.2+cu118", "transformers==4.41.2",  
"https://github.com/Dao-AILab/flash-  
attention/releases/download/v2.5.8/flash_attn-  
2.5.8+cu118torch2.0cxx11abiFALSE-cp311-cp311-linux_x86_64.whl"],  
    input_example=input_example,  
    registered_model_name=registered_model_name)
```

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Use custom Python libraries with Model Serving

In this article, you learn how to include custom libraries or libraries from a private mirror server when you log your model, so that you can use them with [Model Serving](#) model deployments. You should complete the steps detailed in this guide after you have a trained ML model ready to deploy but before you create a Databricks [Model Serving endpoint](#).

Model development often requires the use of custom Python libraries that contain functions for pre- or post-processing, custom model definitions, and other shared utilities. In addition, many enterprise security teams encourage the use of private PyPi mirrors, such as Nexus or Artifactory, to reduce the risk of [supply-chain attacks](#). Databricks offers [native support](#) for installation of custom libraries and libraries from a private mirror in the Databricks workspace.

Requirements

- MLflow 1.29 or higher
- Restrict outbound network access from Model Serving endpoints by configuring network policies. See [Validate with model serving](#).

Option 1: Use a private package repository

Use Option 1 if your organization uses a private PyPI mirror (such as Nexus or Artifactory). Workspace admins can configure it as the default package r

the workspace. Model Serving automatically uses this workspace-level configuration when building your model environment.

To set up a private package repository, see [Configure default Python package repositories](#).

Once configured, proceed to [Serve your model](#).

Option 2: Package custom libraries as wheel files

Use Option 2 if a private PyPI mirror is not accessible, or if you have custom libraries that are not available in any package repository. You can package them as Python wheel files and include them when logging your model.

Step 1: Upload your dependency file

Databricks recommends that you upload your dependency file to Unity Catalog [volumes](#). Alternatively, you can upload it to [Databricks File System \(DBFS\)](#) using the Databricks UI.

To ensure your library is available to your notebook, you need to install it using `%pip`. Using `%pip` installs the library in the current notebook and downloads the dependency to the cluster.

Step 2: Log the model with a custom library

After you install the library and upload the Python wheel file to either Unity Catalog volumes or DBFS, include the following code in your script. In the `extra_pip_requirements` specify the path of your dependency file.

Python

```
mlflow.sklearn.log_model(model, "sklearn-model", extra_pip_requirements=
["/volume/path/to/dependency.whl"])
```

 Ask Genie

For DBFS, use the following:

Python

```
mlflow.sklearn.log_model(model, "sklearn-model", extra_pip_requirements=
["/dbfs/path/to/dependency.whl"])
```

If you have a custom library, you must specify all custom Python libraries associated with your model when you configure logging. You can do so with the `extra_pip_requirements` or `conda_env` parameters in `log_model()`.

❗ IMPORTANT

If using DBFS, be sure to include a forward slash, `/`, before your `dbfs` path when logging `extra_pip_requirements`. Learn more about DBFS paths in [Work with files on Databricks](#).

Python

```
from mlflow.utils.environment import _mlflow_conda_env

mlflow.pyfunc.log_model(
    name="model",
    python_model=MyModel(),
    extra_pip_requirements=["/volumes/path/to/dependency"],
)
```

If your custom library is stored somewhere other than a volume or DBFS, you can specify its location using the `code_paths` parameter, and pass `"code/<wheel-file-name>.whl"` in the `extra_pip_requirements` parameter.

Python

```
mlflow.pyfunc.log_model(
    name="model",
    python_model=MyModel(),
    code_paths=["/path/to/dependency.whl"], # This will be logged as
`code/dependency.whl`
    extra_pip_requirements=["code/dependency.whl"],
)
```

 Ask Genie

Step 3: Update MLflow model with Python wheel files

MLflow provides the `add_libraries_to_model()` utility to log your model with all of its dependencies pre-packaged as Python wheel files. This packages your custom libraries alongside the model in addition to *all* other libraries that are specified as dependencies of your model. This guarantees that the libraries used by your model are exactly the ones accessible from your training environment.

In the following example, `model_uri` references the Unity Catalog model registry using the syntax `models:/<uc-model>/<model-version>`. To reference the workspace model registry (legacy) use, `models:/<model-name>/<model-version>`.

When you use the model registry URI, this utility generates a new version under your existing registered model.

Python

```
import mlflow.models.utils
mlflow.models.utils.add_libraries_to_model(<model-uri>)
```

Serve your model

When a new model version with the packages included is available in the model registry, you can add this model version to an endpoint with [Model Serving](#).

Troubleshoot package installation

If your model deployment fails during the build phase, you can review the build logs to identify package installation issues.

1. Go to the **Serving** page in your Databricks workspace.
2. Click your endpoint name to open the endpoint details.
3. Click the **Logs** tab.
4. Select the failed version from the drop-down menu.

5. Click **Build logs**.

Review the error messages to identify the issue.

After resolving the issue, create a new deployment or update your endpoint to trigger a new build.

Troubleshoot private package repository

If you are using a private package repository, common issues include:

- **Missing packages:** The package is not available in your configured repository. Add the required package to your private repository.
- **Connection issues:** Model Serving can't reach your package repository. Verify network connectivity and firewall rules.
- **Authentication failures:** The credentials configured for your repository are not valid or expired. Update the secrets in your workspace configuration.

Serverless notebooks use the same default package repository configured for your workspace. You can use a notebook to test connectivity, authentication, and package availability by installing the requirements from your model's `requirements.txt` file before deploying to Model Serving.

Python

```
import mlflow
import subprocess
import sys

# Step 1: Set your model details
catalog = "<your_catalog>"
schema = "<your_schema>"
model_name = "<your_model>"
version = <your_version>

# Step 2: Download the model's requirements.txt
full_model_name = f"{catalog}.{schema}.{model_name}"
requirements_uri =
f"models://{full_model_name}/{version}/requirements.txt"

print(f"Downloading artifacts from: {requirements_uri}")
local_path = mlflow.artifacts.download_artifacts(requirements_uri)
```

 Ask Genie

```
# Step 3: Print the requirements
with open(local_path, "r") as f:
    print(f.read())

# Step 4: Install the requirements using the workspace's default package
repository
print(f"Installing requirements from {local_path}...")
subprocess.check_call([sys.executable, "-m", "pip", "install", "-r",
local_path])
print("Installation complete!")
```

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 7, 2026**

Package custom artifacts for Model Serving

This article describes how to ensure your model's file and artifact dependencies are available on your [Deploy models using Model Serving](#) endpoint.

Requirements

MLflow 1.29 and above

Package artifacts with models

When your model requires files or artifacts during inference, you can package them into the model artifact when you log the model.

If you are working with Databricks notebooks, a common practice is to have these files reside in [Unity Catalog volumes](#). Models are also sometimes configured to download artifacts from the internet (such as HuggingFace Tokenizers). Real-time workloads at scale perform best when all required dependencies are statically captured at deployment time. For this reason, Model Serving requires that Unity Catalog volumes artifacts are packaged into the model artifact itself using MLflow interfaces. Network artifacts loaded with the model should be packaged with the model whenever possible.

With the MLflow command `log_model()` you can log a model and its dependent artifacts with the `artifacts` parameter.

Python

 Ask Genie

```
mlflow.pyfunc.log_model(
    ...
    artifacts={'model-weights':
"/Volumes/catalog/schema/volume/path/to/file", "tokenizer_cache":
"./tokenizer_cache"},
    ...
)
```

In PyFunc models, these artifacts' paths are accessible from the `context` object under `context.artifacts`, and they can be loaded in the standard way for that file type.

For example, in a custom MLflow model:

Python

```
class ModelPyfunc(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        self.model = torch.load(context.artifacts["model-weights"])
        self.tokenizer =
transformers.BertweetTokenizer.from_pretrained("model-base",
local_files_only=True, cache_dir=context.artifacts["tokenizer_cache"])
    ...
```

After your files and artifacts are packaged within your model artifact, you can serve your model to a [Model Serving endpoint](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Deploy Python code with Model Serving

This article describes how to deploy your customized Python code with [Model Serving](#). The example in this article focuses on providing guidance for adding preprocessing and postprocessing logic to your model and deploying it.

MLflow's Python function, `pyfunc`, provides flexibility to deploy any piece of Python code or any Python model. The following are example scenarios where you might want to use the guide.

- Your model requires preprocessing before inputs can be passed to the model's predict function.
- Your model framework is not natively supported by MLflow.
- Your application requires the model's raw outputs to be post-processed for consumption.
- The model itself has per-request branching logic.
- You are looking to deploy fully custom code as a model.

Construct a custom MLflow Python function model

MLflow offers the ability to log Python code with the [custom Python models format](#).

There are two required functions when packaging arbitrary python code with MLflow:

- `load_context` – anything that needs to be loaded just one time for the model to operate should be defined in this function. This is critical so that the

minimize the number of artifacts loaded during the `predict` function, which speeds up inference.

- `predict` – this function houses all the logic that is run every time an input request is made.

NOTE

Prior to deploying your custom code as a model, it is beneficial to verify that the model is capable of being served. See the MLflow documentation for how you can use `mlflow.models.predict` to [validate models before deployment](#).

Log your Python function model

Even though you are writing your model with custom code, it is possible to use shared modules of code from your organization. With the `code_path` parameter, authors of models can log full code references that load into the path and are usable from other custom `pyfunc` models.

For example, if a model is logged with:

Python

```
mlflow.pyfunc.log_model(CustomModel(), "model", code_path =
["preprocessing_utils/"])
```

Code from the `preprocessing_utils` is available in the loaded context of the model. The following is an example model that uses this code.

Python

```
class CustomModel(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        self.model = torch.load(context.artifacts["model-weights"])
        from preprocessing_utils.my_custom_tokenizer import
CustomTokenizer
        self.tokenizer =
CustomTokenizer(context.artifacts["tokenizer_cache"])

    def format_inputs(self, model_input):
```

 Ask Genie

```

# insert some code that formats your inputs
pass

def format_outputs(self, outputs):
    predictions = (torch.sigmoid(outputs)).data.numpy()
    return predictions

def predict(self, context, model_input):
    model_input = self.format_inputs(model_input)
    outputs = self.model.predict(model_input)
    return self.format_outputs(outputs)

```

Serve your model

After you log your custom `pyfunc` model, you can register it to Unity Catalog or Workspace Registry and serve your model to a [Model Serving endpoint](#).

Notebook example

The following notebook example demonstrates how to customize model output when the raw output of the queried model needs to be post-processed for consumption.

Customize model serving output with MLflow PyFunc notebook

[Open notebook in new tab](#)

 Copy link for import

Customizing model serving output with MLflow PyFunc

This notebook demonstrates how to customize the output of a served model when the raw outputs of the queried model needs to be post-processed for consumption.

```

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

```

Expand notebook ▼

 Ask Genie

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **May 11, 2026**

Optimize Model Serving endpoints for production

Learn how to optimize Model Serving endpoints for production workloads that require high throughput, low latency, and reliable performance.

Optimization strategies fall into three categories:

- **Endpoint optimizations:** Configure endpoint infrastructure for better performance
- **Model optimizations:** Improve model efficiency and throughput
- **Client optimizations:** Optimize how clients interact with serving endpoints

When to optimize your endpoint

Consider optimizing your Model Serving endpoint when you encounter any of the following scenarios:

- **High query volume:** Your application sends more than 50k queries per second (QPS) to a single endpoint
- **Latency requirements:** Your application requires sub-100ms response times
- **Scaling bottlenecks:** Endpoints experience queuing or return HTTP 429 errors during traffic spikes
- **Cost optimization:** You want to reduce serving costs while maintaining performance targets
- **Production preparation:** You're preparing to move from development to production workloads

Infrastructure optimizations

Infrastructure optimizations improve network routing, scaling behavior, and compute capacity.

Route optimization

Route optimization provides the most significant infrastructure improvement for high-throughput workloads. When you enable route optimization on an endpoint, Databricks Model Serving improves the network path for inference requests, resulting in faster, more direct communication between clients and models.

Performance benefits:

Feature	Standard endpoint limit	Route optimized endpoint limit
Queries per second (QPS) per workspace	200	50,000+ (contact Databricks for higher limits)
Client concurrency per workspace	192-1024 (varies by region)	No explicit limit (limited by provisioned concurrency)
Endpoint provisioned concurrency per served entity	1,024	1,024 (contact Databricks for higher limits)

When to use route optimization:

- Workloads requiring more than 200 QPS
- Applications with strict latency requirements (sub-50ms overhead)
- Production deployments serving multiple concurrent users

ⓘ IMPORTANT

Route optimization is only available for custom model serving endpoints. Foundation Model APIs and external models do not support route optimization.

OAuth tokens are required for authentication; personal access tokens are not supported.

See [Route optimization on serving endpoints](#) for setup instructions and [Query route-optimized serving endpoints](#) for querying details.

Provisioned concurrency

Provisioned concurrency controls how many simultaneous requests your endpoint can process. Configure provisioned concurrency based on your expected QPS and latency requirements.

Configuration guidelines:

- **Minimum concurrency:** Set high enough to handle baseline traffic without queuing
- **Maximum concurrency:** Set high enough to accommodate traffic spikes while controlling costs
- **Autoscaling:** Enable autoscaling to dynamically adjust capacity based on demand

Calculate required concurrency:

$$\text{Required Concurrency} = \text{Target QPS} \times \text{Average Latency (seconds)}$$

For example, if your target is 100 QPS with 200ms average latency:

$$\text{Required Concurrency} = 100 \times 0.2 = 20$$

Use [load testing](#) to measure actual latency and determine optimal concurrency settings.

Instance types

Choose instance types based on your model's compute requirements:

Instance type	Best for	Trade-offs
CPU (Small, Medium, Large)	Lightweight models, simple inference logic	Lower cost, slower for compute-intensive models
GPU (Small, Medium, Large)	Large models, complex computations, image/video processing	Higher cost, optimal performance for deep learning

TIP

Start with CPU instances for development and testing. Switch to GPU instances only if you observe high inference latency or your model requires specialized compute (such as deep learning operations).

Model optimizations

Model optimizations improve inference speed and resource efficiency.

Model size and complexity

Model Size and Complexity: Smaller, less complex models generally lead to faster inference times and higher QPS. Consider techniques like model quantization or pruning if your model is large.

Batching

If your application can send multiple requests in a single call, enable batching at the client side. This can significantly reduce the overhead per prediction.

Pre-processing and post-processing optimization

Offload complex pre-processing and post-processing from serving endpoints to reduce load on inference infrastructure.  Ask Genie

Client-side optimizations

Client-side optimizations improve how applications interact with serving endpoints.

Connection pooling

Connection pooling reuses existing connections instead of creating new connections for each request, significantly reducing overhead.

- Use the Databricks SDK, which automatically implements connection pooling best practices
- If using custom clients, implement connection pooling yourself.

Error handling and retry strategies

Implement robust error handling to gracefully handle temporary failures, especially during autoscaling events or network disruptions.

Payload size optimization

Minimize request and response payload sizes to reduce network transfer time and improve throughput.

Measure and improve performance

Performance monitoring

Monitor endpoint performance using the tools provided by Model Serving:

Metric	What it measures	Target	Action if exceeded
Latency (P50, P90, P99)	Response time for requests	Application-dependent (typically <100–500ms)	Check for queuing, optimize model or client
Throughput (QPS)	Requests completed per second	Workload-dependent	Enable route optimization, increase provisioned concurrency
Error rate	Percentage of failed requests	<1%	Review service logs, check for capacity issues
Queue depth	Requests waiting for processing	0 (no queuing)	Increase provisioned concurrency or enable autoscaling
CPU/Memory usage	Resource utilization	<80%	Scale up instance type or increase concurrency

See [Monitor model quality and endpoint health](#) for detailed monitoring guidance and [Track and export serving endpoint health metrics to Prometheus and Datadog](#) for exporting metrics to observability tools.

Load testing

Load testing measures endpoint performance under realistic traffic conditions and helps you:

- Determine optimal provisioned concurrency settings
- Identify performance bottlenecks
- Validate latency and throughput requirements

- Understand the relationship between client concurrency and server concurrency

See [Load testing for serving endpoints](#).

Troubleshoot common performance issues

Queuing

Model Serving supports autoscaling to adjust capacity based on traffic patterns. However, sudden traffic surges can cause queuing because autoscaling requires time to detect increased load and provision additional capacity. During this period, incoming requests may temporarily exceed available capacity, causing requests to queue.

Queuing occurs when the request rate or concurrency surpasses the endpoint's current processing capacity. This typically happens during sharp traffic spikes, workload bursts, or when the endpoint has insufficient provisioned concurrency. Model Serving endpoints allow temporary queuing to handle bursts, but beyond a defined threshold, the endpoint returns HTTP 429 (Too Many Requests) errors to protect system stability.

Queuing increases latency because queued requests wait before being processed. To minimize queuing:

- Set minimum provisioned concurrency high enough to handle baseline traffic plus typical bursts
- Enable route optimization for higher capacity limits
- Implement retry logic with exponential backoff in your client applications

External API bottlenecks

Models often call external APIs for data enrichment, feature retrieval, or other tasks during inference. These external dependencies can become performance bottlenecks:

- **Latency:** Measure the response time of each external API call. High latency in these calls directly increases overall serving latency and reduces throughput.
- **Throughput limits:** External APIs may impose rate limits or capacity constraints. Exceeding these limits can cause throttling, errors, and performance degradation.
- **Error rates:** Frequent errors from external APIs can trigger retries and increase load on your serving endpoint.
- **Caching:** Implement caching for frequently accessed data from external APIs to reduce the number of calls and improve response times.

Monitor these factors to identify bottlenecks and implement targeted optimizations for high-throughput workloads.

Additional resources

- [Route optimization on serving endpoints](#)
- [Load testing for serving endpoints](#)

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **May 7, 2026**

Configure access to resources from model serving endpoints

This article describes how to configure access to external and private resources from model serving endpoints. Model Serving supports plain text environment variables and secrets-based environment variables using Databricks [secrets](#).

Requirements

For secrets-based environment variables,

- The endpoint creator must have READ access to the Databricks secrets being referenced in the configs.
- You must store credentials like your API key or other tokens as a Databricks secret.

Add plain text environment variables

Use plain text environment variables to set variables that don't need to be hidden. You can set variables in the Serving UI, the REST API or the SDK when you create or update an endpoint.

Serving UI

REST API

WorkspaceClient
SDK

MLflow Deployments SDK

From the Serving UI, you can add an environment variable in **Advanced configurations**:

 Ask Genie

Entity details

Entity

models.default.addmodel

Version

1

Traffic (%)

100

Validate this model's payload and dependencies. [See how here](#).

Compute type

CPU

Compute scale-out

Custom

Minimum concurrency

0

Maximum concurrency

4

☒ Scale to zero

Not recommended for production use. Expect higher latency on the first request as the endpoint scales up.

☐ Enable tracing

Enabling tracing will record inputs, outputs, and parameters from instrumented steps during model inference. Trace data will be written to the (required) configured Inference Table. [Learn more here](#).

Advanced configuration

Environment variables

+ Add environment variables

Instance profile

Select instance profile

Served entity name

addmodel-1

Log feature lookup DataFrames to inference tables

If you have inference tables enabled on your endpoint, you can log your [automatic feature lookup data frame](#) to that inference table using `ENABLE_FEATURE_TRACING`. This requires MLflow 2.14.0 or above.

Set `ENABLE_FEATURE_TRACING` as an environment variable in the Serving UI, REST API or SDK when you create or update an endpoint.

Serving UI

REST API

WorkspaceClient
SDK

MLflow Deployments SDK

-
1. In **Advanced configurations**, select **** + Add environment variables****.
 2. Type `ENABLE_FEATURE_TRACING` as the environment name.
 3. In the field to the right type `true`.

Entity details

Entity

models.default.addmodel

Version

1

Traffic (%)

100

Validate this model's payload and dependencies. [See how here](#).

Compute type

CPU

Compute scale-out

Custom

Minimum concurrency

0

Maximum concurrency

4

☒ Scale to zero

Not recommended for production use. Expect higher latency on the first request as the endpoint scales up.

☐ Enable tracing

Enabling tracing will record inputs, outputs, and parameters from instrumented steps during model inference. Trace data will be written to the (required) configured Inference Table. [Learn more here](#).

Advanced configuration

Environment variables

+ Add environment variables

Instance profile

Select instance profile

Served entity name

addmodel-1

Add secrets-based environment variables

You can securely store credentials using Databricks secrets and reference those secrets in model serving using a secrets-based environment variables. This allows credentials to be fetched from model serving endpoints at serving time.

For example, you can pass credentials to call OpenAI and other external model endpoints or access external data storage locations directly from model serving.

Databricks recommends this feature for deploying [OpenAI](#) and [LangChain](#) MLflow model flavors to serving. It is also applicable to other SaaS models requiring credentials with the understanding that the access pattern is based on using environment variables and API keys and tokens.

Step 1: Create a secret scope

During model serving, the secrets are retrieved from Databricks secrets by the secret scope and key. These get assigned to the secret environment variable names that can be used inside the model.

First, create a secret scope. See [Manage secret scopes](#).

The following are CLI commands:

Bash

```
databricks secrets create-scope my_secret_scope
```

You can then add your secret to a desired secret scope and key as shown below:

Bash

```
databricks secrets put-secret my_secret_scope my_secret_key
```

The secret information and the name of the environment variable can then be passed to your endpoint configuration during endpoint creation or as an update to the configuration of an existing endpoint.

Step 2: Add secret scopes to endpoint configuration

You can add the secret scope to an environment variable and pass that variable to your endpoint during endpoint creation or configuration updates. See [Create custom model serving endpoints](#).

Serving UI

REST API

WorkspaceClient
SDK

MLflow Deployments SDK

From the Serving UI, you can add an environment variable in **Advanced configurations**. The secrets based environment variable must be provided using the following syntax: `{{secrets/scope/key}}`. Otherwise, the environment variable is considered a plain text environment variable.

Entity details

Entity

models.default.addmodel

Version

1

Traffic (%)

100

Validate this model's payload and dependencies. [See how here](#).

Compute type

CPU

Compute scale-out

Custom

Minimum concurrency

0

Maximum concurrency

4

☒ Scale to zero

Not recommended for production use. Expect higher latency on the first request as the endpoint scales up.

☐ Enable tracing

Enabling tracing will record inputs, outputs, and parameters from instrumented steps during model inference. Trace data will be written to the (required) configured Inference Table. [Learn more here](#).

Advanced configuration

Environment variables

+ Add environment variables

Instance profile

Select instance profile

Served entity name

addmodel-1

After the endpoint is created or updated, model serving automatically fetches the secret key from the Databricks secrets scope and populates the environment variable for your model inference code to use.

Notebook example

See the following notebook for an example of how to configure an OpenAI API key for a LangChain Retrieval QA Chain deployed behind the model serving endpoints with secret-based environment variables.

Configure access to resources from model serving endpoints notebook

[Open notebook in new tab](#)

 Copy link for import

Configure access to resources from model serving endpoints

This notebook example demonstrates how you can securely store credentials in a Databricks secrets scope and reference those secrets in model serving. This allows credentials to be fetched at serving time from model serving endpoints.

Requirements

Expand notebook 

Additional resource

- [Add an instance profile to a model serving endpoint](#)

Is this page

as this page helpful?

 Yes

 No

 Send feedback

 Ask Genie

Last updated on **May 7, 2026**

Add an instance profile to a model serving endpoint

This article demonstrates how to attach an instance profile to a model serving endpoint. Doing so allows customers to access any AWS resources from the model permissible by the instance profile. Learn more about [instance profiles](#).

Requirements

- [Create an instance profile](#).
- [Add an instance profile to Databricks](#).
 - If you have an instance profile already configured for serverless SQL, be sure to change the access policies so that your models have the right access policy to your resources.
- Model serving endpoints run on serverless compute. Your instance profile's IAM role must have a trust relationship configured for serverless compute. If you see the error "IAM role does not have the required trust relationship," see [Confirm or set up an AWS instance profile to use with your serverless SQL warehouses](#) for setup instructions.

Add an instance profile during endpoint creation

When you [create a model serving endpoint](#) you can add an instance profile to the endpoint configuration.

NOTE

The endpoint creator's permission to an instance profile is validated at endpoint creation time.

- From the Serving UI, you can add an instance profile in **Advanced configurations**:

Entity details ✕

Entity	Version	Traffic (%)
 main.lineage_demo.example_model 	3 	100

Compute Type

CPU 

Compute Scale-out



☒ Scale to zero

Advanced configuration

Environment Variables

 Add Environment Variables

Instance Profile

Select Instance Profile 

Served entity name

example_model-3

- For programmatic workflows, use the `instance_profile_arn` field when you create an endpoint to add an instance profile.

Bash

POST /api/2.0/serving-endpoints

{

 Ask Genie

```

"name": "feed-ads",
"config":{
"served_entities": [{
  "entity_name": "ads1",
  "entity_version": "1",
  "workload_size": "Small",
  "scale_to_zero_enabled": true,
  "instance_profile_arn": "arn:aws:iam::<aws-account-id>:instance-
profile/<instance-profile-name-1>"
}]
}
}

```

Update an existing endpoint with an instance profile

You can also update an existing model serving endpoint configuration with an instance profile with the `instance_profile_arn` field.

Bash

```
PUT /api/2.0/serving-endpoints/{name}/config
```

```

{
  "served_entities": [{
    "entity_name": "ads1",
    "entity_version": "2",
    "workload_size": "Small",
    "scale_to_zero_enabled": true,
    "instance_profile_arn": "arn:aws:iam::<aws-account-id>:instance-
profile/<instance-profile-name-2>"
  }]
}

```

Limitations

The following limitations apply:

- STS temporary security credentials are used to authenticate data access. It can't bypass any network restriction.

- If customers edit the instance profile IAM role from the **Settings** of the Databricks UI, endpoints running with the instance profile continue to use the old IAM role until the endpoint updates.
- If customers delete an instance profile from the **Settings** of the Databricks UI and that profile is used in running endpoints, the running endpoint is not impacted.

For general model serving endpoint limitations, see [Model Serving limits and regions](#).

Additional resources

- [Look up features](#) using the same instance profile that you added to the serving endpoint.
- [Configure access to resources from model serving endpoints](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Load testing for serving endpoints

This article walks you through the essential process of load testing your Model Serving endpoints to ensure they can handle production workloads effectively. It also provides practical examples, real-world analogies, and step-by-step instructions using the Locust load testing framework, to demonstrate how to measure key performance metrics like requests per second and concurrency limits, so you can size your endpoints correctly and confidently deploy models for your business needs.

💡 TIP

Load testing is an essential component of production optimization. For a comprehensive guide to optimization strategies including infrastructure, model, and client-side optimizations, see [Optimize Model Serving endpoints for production](#).

What is a load test?

A load test is a test which simulates real world usage of Model Serving endpoints ensuring they meet your production requirements, like latency or requests per second. A load test measures your endpoint's performance under different levels of traffic, helping you size your endpoint correctly so as to not cause delays.

Picture this: It's 8:00 AM on a weekday, and a popular café in the heart of the city is just opening. The aroma of fresh coffee fills the air. The barista is prepped, machines warmed up, and the line of caffeine-deprived customers is already forming.

At first, things run smoothly. A couple of customers step up, place their order, and the barista begins preparing their drinks. Some drinks take 30 seconds, other

minute — depending on the complexity. The barista juggles multiple orders with practiced efficiency.

But soon, more people arrive. Five customers turn into ten, then twenty. Each is placing an order, waiting for their drink, and maybe chatting a bit at the counter. The barista is now under pressure. Even with a second barista jumping in, the system starts to strain — the line grows, orders pile up, and customers begin waiting longer.

Now imagine you're trying to measure how many drinks the café can serve per minute before customers start to leave frustrated. That's **load testing**.

In this analogy:

- Each customer is a **client** sending a request.
- The barista(s) represent your **server** that processes model inferences one by one or in parallel.
- The time to take an order and serve the drink is the **model implementation** time.
- Delays in talking, paying, or finding the order are your **network overhead**.
- More customers arriving at once is **client concurrency**.
- Adding more baristas or more machines is like increasing your **server concurrency**.

As in any good café, there's a limit to how much the staff can handle at once. If you don't plan ahead — say, by bringing in more baristas during peak hours — people leave unhappy. The same goes for your systems under load. Load testing helps you identify where the bottlenecks are, how much traffic your system can handle, and what changes you need to make for smoother service.

Before you run a load test on your endpoint, you need to:

- Understand the components and concepts related to load testing.
- Decide and define your production requirements.
- Find a representative payload for the load testing framework to use when benchmarking your endpoint.

Load testing concepts and definitions

The following table defines load testing related concepts:

Concept	Description
Client concurrency (number of clients)	The total number of clients that simultaneously send requests in parallel to an endpoint. These are your customers to the cafe in the above example. Production: The total instances of clients sending traffic to the model serving endpoint. Load test: In Locust, this is the number of users created that send traffic to the model serving endpoint being load tested.
Endpoint concurrency	The number of inference processes or model instances that can handle incoming requests. Can also be expressed as the number of requests that can be simultaneously handled by your endpoint. It is the number of baristas in the above example. Model Serving: Model serving endpoints can be configured for compute scale out sizes. For example, using the <code>Small</code> size of CPU endpoints creates four instances of your model on the endpoint.
Latency	The time (in milliseconds) for a full round-trip request to complete. A measure of the total time after the client has sent a request until the response has been received, inclusive of endpoint run time and network latency.
PXX (P50,P90,P99) latency	The latency (in milliseconds) for which XX percentile of requests has finished faster than. For example, a P90 of 30ms means that 90% of all requests have finished in 30ms or less.
Requests per second (RPS)	The number of requests completed per second. Completed means that a response is returned from the endpoint to the client.  Ask Genie

Latency requirements

Based upon your business and customer requirements, define the ideal performance of your system. On a model serving endpoint, latency includes the round trip time that a client experiences when sending data for inference. This includes networking latency as well as inference time. It is important to ensure that your requirements are realistic. For example, if your model takes 15ms to perform inference when loaded into memory, you need to allow additional time for networking latency when served on a model serving endpoint.

How do RPS, latency, and concurrency relate?

A business has some defined criteria for the success of their system. For ML systems, business criteria is generally high quality results, low latency, and high throughput. The quality of results is specifically related to the model itself, while end-to-end latency and throughput are traits of the serving system. End-to-end latency consists of the model execution time and the network overhead. Throughput (or requests or queries per second) is inversely related to latency and directly related to concurrency.

- The more concurrency the higher the throughput.
- The higher the latency the lower the throughput.

Generally, there is an optimal ratio of client side concurrency to server side concurrency for any given application. As an example, “how many burgers can a line chef work on simultaneously”. Because there may be many shared steps in the cooking process, the line chef might be able to more optimally cook four hamburgers at the same time rather than cooking only one at a time. There is a limit to this parallelization, at some point the act of performing many parallel acts adds too much latency, like if the chef needs to add cheese to 1000 burgers.

One of the central goals of a load test is to determine the optimal ratio for your application. The optimal ratio maximizes RPS, meet your latency requirements, and avoid queuing. This ratio can be used to accurately size your endpoint to meet your most demanding loads.

If the endpoint is unable to process requests fast enough, a line begins to form. This is called a queue. The forming of a queue typically results in much longer end-to-end latency as there is now additional time each request spends waiting before being

processed. If requests continue to arrive faster than requests can be processed, the queue grows, which further increases latency. For this reason, it is important to understand what sort of peak demand your endpoint may experience and ensure it is sized correctly with a load test.

Load test scenario examples

In load testing, as well as real-world systems, the relationship between client concurrency, server concurrency, and latency is dynamic and interdependent. Let's see this relationship with a simple example:

Scenario 1: Simple setup

In this setup, a single client sends requests sequentially — it waits for a response before issuing the next request. Since the total time per request is the sum of model execution and overhead latency (40 ms + 10 ms), the measured end to end latency is 50 ms.

- Number of clients: 1
- Provisioned concurrency: 1
- Overhead latency: 10 ms
- Model execution time 40 ms

As a result, the client completes one request every 50 ms, which equates to 20 requests per second or queries per second.

Scenario 2: Increase provisioned concurrency

In this case, you have double the provisioned concurrency and a single client making requests sequentially. This means that the total latency is still 50 ms (40ms + 10ms), and the system is only seeing 20 requests per second (QPS).

- Number of clients: 1
- Provisioned concurrency: **1 → 2**
- Overhead latency: 10 ms
- Model execution time 40 ms

Scenario 3: Add another client to the system.

Now you have two clients making requests to an endpoint with two provisioned concurrency. In this case, each client's request can be independently processed by the endpoint simultaneously (assuming perfect load balancing). So while the total latency is still 50 ms (40ms + 10ms), the system observes 40 requests per seconds, since each client sends 20 qps.

- Number of clients: **1 -> 2**
- Provisioned concurrency: 2
- Overhead latency: 10 ms
- Model execution time 40 ms

Increasing the provisioned concurrency and the number of clients making requests increases the total QPS observed in your system, without a penalty on latency.

Scenario 4: Lets reduce the provisioned concurrency

In this last scenario, the number of clients is greater than the provisioned concurrency. This setup introduces another variable, queuing, in the system and its effect on QPS and latency.

- Number of clients: 2
- Provisioned concurrency: **2 -> 1**
- Overhead latency: 10 ms
- Model execution time: 40 ms

Here, you have two clients making requests simultaneously. The endpoint, however, can only process a single request at a time. This forces the second request to wait before the first request has been processed. The waiting, or more correctly, queuing of the second request can adversely affect the latency of the system. Assuming that the server allows queuing (enabled by default in Databricks Model Serving), the second client sees a latency of 90 ms: 10 ms (network overhead) + 40 ms (queuing wait time) + 40 ms (model execution time). This is significantly worse than the 50 ms seen before.

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **May 11, 2026**

Manage model serving endpoints

This article describes how to manage model serving endpoints using the **Serving** UI and REST API. See [Serving endpoints](#) in the REST API reference.

To create model serving endpoints use one of the following:

- [Create custom model serving endpoints](#).
- [Create foundation model serving endpoints](#).

Get the status of the model endpoint

You can check the status of an endpoint using the **Serving** UI or programmatically using the REST API, Databricks Workspace Client, or MLflow Deployments SDK.

Endpoint statuses can be `Ready`, `Ready (Update failed)`, `Not ready (Updating)`, `Not ready (Update failed)`, or `Not ready (Stopped)`. Readiness refers to whether or not an endpoint can be queried. Updated failed indicates the latest change to the endpoint was unsuccessful. Stopped means the endpoint was stopped.

UI REST API Databricks Workspace Client MLflow Deployments SDK

The **Serving endpoint state** indicator at the top of an endpoint's details page:

Serving endpoints >

my-serving-endpoint

Ready <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/my-serving-endpoint/invocations>

Gateway

Usage monitoring: [system.serving.endpoint_usage](#) Rate limits: Not enabled

Dimension table: [system.serving.served_entities](#)

Inference tables: Not enabled

Active configuration

Entity	Version	Name	State	Compute	Traffic (%)	Description
			Ready	CPU, Custom 0-4 concurrency (0-4 DBU)	100	No description

About this endpoint

Created by

Route Not enabled

optimization

Tags

Add

Serverless usage policy

Description

No description

Serving endpoints [Send feedback](#) [Create serving endpoint](#)

Experimenting with LLMs? Try managed LLM models or securely connect to external providers like OpenAI!

Integrate coding agents
Securely connect to coding agents in Databricks
[Get Started](#)

GPT-5
Chat · Databricks-hosted Model
[Use](#) [Copy](#)

GPT-5 Mini
Chat · Databricks-hosted Model
[Use](#) [Copy](#)

OpenAI GPT-5
Chat · External Model
[Configure](#)

GPT-5 Nano
Chat · Databricks-hosted Model
[Use](#)

Filter serving endpoints by name ☐ Owned by me

Name	State	Served entities	Tags	Task	Created by	Last modified
	Ready		(v 2)			15 hours ago
my-serving-endpoint	Not ready (Stopped)		(v 1)			1 day ago
	Ready		(v 1)			3 days ago
	Ready		(v 1)			6 days ago

Stop a model serving endpoint

You can temporarily stop a model serving endpoint and start it later. When an endpoint is stopped:

- The resources provisioned for it are shut down.
- The endpoint is not able to serve queries until it is started again.
- Only endpoints that serve [custom models](#) and have no in-progress updates can be stopped.
- Stopped endpoints do not count against the resource quota.
- Queries sent to a stopped endpoint return a 400 error.

Stop an endpoint

UI REST API

Click **Stop** in the upper-right corner.

Serving endpoints > **my-serving-endpoint** Alerts Permissions Edit **Stop** Use

Ready <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/my-serving-endpoint/invocations>

Gateway Edit AI Gateway

Usage monitoring: [system.serving.endpoint_usage](#) Rate limits: Not enabled

Dimension table: [system.serving.served_entities](#)

Inference tables: Not enabled

Active configuration

Entity	Version	Name	State	Compute	Traffic (%)
			Ready	CPU, Custom 0-4 concurrency (0-4 DBU)	100

About this endpoint

Created by

Route optimization Not enabled

Tags Add

Serverless usage policy

Description No description

Start an endpoint

Starting an endpoint creates a new config version with the same properties as the existing stopped config.

When you are ready to start a stopped model serving endpoint:

UI REST API

Click **Start** in the upper-right corner.

Serving endpoints > **my-serving-endpoint** Alerts Permissions Edit **Start** Use

Not ready (Stopped) <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/my-serving-endpoint/invocations>

Gateway Edit AI Gateway

Usage monitoring: [system.serving.endpoint_usage](#) Rate limits: Not enabled

Dimension table: [system.serving.served_entities](#)

Inference tables: Not enabled

Active configuration

Entity	Version	Name	State	Compute	Traffic (%)
			Ready	CPU, Custom 0-4 concurrency (0-4 DBU)	100

About this endpoint

Created by

Route optimization Not enabled

Tags Add

Serverless usage policy

Description No description

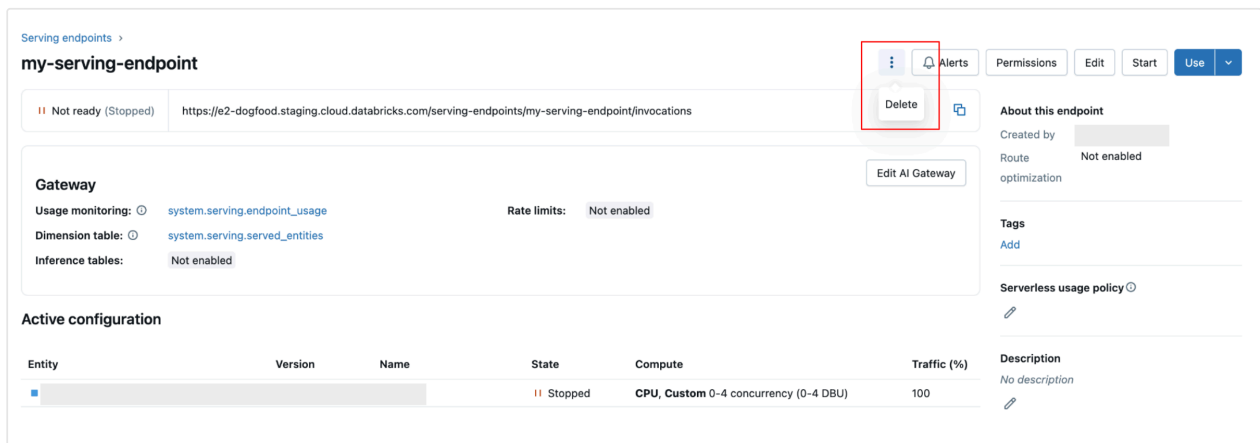
Delete a model serving endpoint

Deleting an endpoint disables usage and deletes all data associated with the endpoint. You cannot undo deletion.

UI REST API MLflow Deployments SDK

Ask Genie

Click the kebab menu at the top and select **Delete**.



Debug a model serving endpoint

Two types of logs are available to help debug issues with endpoints:

- **Model server container build logs:** Generated during endpoint initialization when the container is being created. These logs capture the setup phase including downloading the model, installing dependencies, and configuring the runtime environment. Use these logs to debug why an endpoint failed to start or is stuck during deployment.
- **Model server logs:** Generated during runtime when the endpoint is actively serving predictions. These logs capture incoming requests, model inference execution, runtime errors, and application-level logging from your model code. Use these logs to debug issues with predictions or investigate query failures.

Both log types are also accessible from the **Endpoints** UI in the **Logs** tab.

Get container build logs

For the **build logs** for a served model you can use the following request. See [Debugging guide for Model Serving](#) for more information.

Bash

```
GET /api/2.0/serving-endpoints/{name}/served-models/{served-model-name}/build-logs
```

```
{  
  "config_version": 1 // optional  
}
```

Get model server logs

For the **model server** logs for a serve model, you can use the following request:

Bash

```
GET /api/2.0/serving-endpoints/{name}/served-models/{served-model-name}/logs
```

```
{  
  "config_version": 1 // optional  
}
```

Manage permissions on a model serving endpoint

You must have at least the CAN MANAGE permission on a serving endpoint to modify permissions. For more information on the permission levels, see [Serving endpoint ACLs](#).

Get the list of permissions on the serving endpoint.

UI Databricks CLI

Click the Permissions button at the top right of the UI.

The screenshot shows the Databricks UI for a serving endpoint named 'my-serving-endpoint'. At the top right, there are buttons for 'Alerts', 'Permissions' (highlighted with a red box), 'Cancel update', and 'Use'. Below this, the endpoint status is 'Not ready (Updating)' with a URL: 'https://[redacted].databricks.com/serving-endpoints/my-serving-endpoint/invocations'. The 'Gateway' section shows 'Usage monitoring' as 'system.serving.endpoint_usage', 'Dimension table' as 'system.serving.served_entities', and 'Inference tables' as 'Not enabled'. The 'Rate limits' section shows 'Not enabled'. On the right sidebar, there is an 'About this endpoint' section with 'Created by' and 'Route optimization' (both 'Not enabled'), and a 'Tags' section with 'Add' and 'Ask Genie'. At the bottom right, there is a 'Serverless usage policy' link.

You can also modify serving endpoint permissions using the [Permissions API](#).

Add a serverless usage policy for a model serving endpoint

⚠️ PREVIEW

This feature is in [Public Preview](#) and is not available for serving endpoints that serve [External models](#).

Serverless usage policies allow your organization to apply custom tags on serverless usage for granular billing attribution. If your workspace uses serverless usage policies to attribute serverless usage, you can add a serverless usage policy to your model serving endpoints. See [Attribute usage with serverless usage policies](#).

During model serving endpoint creation, you can select your endpoint's serverless usage policy from the **Budget policy** menu in the Serving UI. If you have a serverless usage policy assigned to you, all endpoints that you create are assigned that serverless usage policy, even if you do not select a policy from the **Budget policy** menu.

Serving endpoints >

Create serving endpoint

Experimenting with LLMs? Try pay-per-token Foundation Model APIs!

General

Name

Endpoint name cannot be changed after creation.

URL preview: <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/>

Served entities

+ Add served entity

AI Gateway Preview >

Monitor and secure endpoints. You can configure these options later. Please refer to the [documentation](#) for more details. [Billing](#) will begin at a later date.

Route optimization

☐ Enable route optimization

Recommended for high throughput or low latency workloads. See [here](#) for more information. Route optimization cannot be changed after endpoint creation.

Tags >

Optional. You can configure tags later

Budget policy

Optional. Tags associated with the budget policy are automatically propagated to the serving endpoint's billing logs. You can configure the budget policy later

Available budget policies

Summary

Served entities

Not configured

Tags

Not configured

Budget policy tags

Not configured

AI Gateway

Not configured

If you have **MANAGE** permissions for an existing endpoint, you can edit and add a serverless usage policy to that endpoint from the **Endpoint details** page in the UI.

[Serving endpoints](#) >

test-budget

Serving endpoint state: Ready

Created by:

URL: <https://e2-dogfood.staging.cloud.databricks.com/serving-endpoints/test-budget/invocations>

Tags: foo: bar

Budget policy : test tag 1: value test tag 2: value 2

Inference table: Not enabled

Route optimization: Not enabled

Active configuration

Entity	Version	Name	State	Compute
models.default.addmodel	Version 1	addmodel-1	Ready	CPU, Small 4 concurrency (4 DBU)

NOTE

If you've been assigned a serverless usage policy, your existing endpoints are not automatically tagged with your policy. You must manually update existing endpoints if you want to attach a serverless usage policy to them.

Get a model serving endpoint schema

PREVIEW

Support for serving endpoint query schemas is in [Public Preview](#). This functionality is available in [Model Serving regions](#).

A serving endpoint query schema is a formal description of the serving endpoint using the standard OpenAPI specification in JSON format. It contains information about the endpoint including the endpoint path, details for querying the endpoint like the request and response body format, and data type for each field. This information can be helpful for reproducibility scenarios or when you need information about the endpoint, but you are not the original endpoint creator or owner.

To get the model serving endpoint schema, the served model must have a model signature logged and the endpoint must be in a **READY** state.

The following examples demonstrate how to programmatically get the model serving endpoint schema using the REST API. For feature serving endpoint schemas, see [Feature Serving endpoints](#). Ask Genie

The schema returned by the API is in the format of a JSON object that follows the OpenAPI specification.

Bash

```
ACCESS_TOKEN="<endpoint-token>"
ENDPOINT_NAME="<endpoint name>"

curl "https://example.databricks.com/api/2.0/serving-
endpoints/$ENDPOINT_NAME/openapi" -H "Authorization: Bearer
$ACCESS_TOKEN" -H "Content-Type: application/json"
```

Schema response details

The response is an OpenAPI specification in JSON format, typically including fields like `openapi`, `info`, `servers` and `paths`. Since the schema response is a JSON object, you can parse it using common programming languages, and generate client code from the specification using third-party tools. You can also visualize the OpenAPI specification using third-party tools like Swagger Editor.

The main fields of the response include:

- The `info.title` field shows the name of the serving endpoint.
- The `servers` field always contains one object, typically the `url` field which is the base url of the endpoint.
- The `paths` object in the response contains all supported paths for an endpoint. The keys in the object are the path URL. Each `path` can support multiple formats of inputs. These inputs are listed in the `oneOf` field.

The following is an example endpoint schema response:

JSON

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "example-endpoint",
    "version": "2"
  },
```

```

"servers": [{ "url": "https://example.databricks.com/serving-
endpoints/example-endpoint" }],
"paths": {
  "/served-models/vanilla_simple_model-2/invocations": {
    "post": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "oneOf": [
                {
                  "type": "object",
                  "properties": {
                    "dataframe_split": {
                      "type": "object",
                      "properties": {
                        "columns": {
                          "description": "required fields: int_col",
                          "type": "array",
                          "items": {
                            "type": "string",
                            "enum": ["int_col", "float_col",
"string_col"]
                          }
                        }
                      },
                    },
                    "data": {
                      "type": "array",
                      "items": {
                        "type": "array",
                        "prefixItems": [
                          {
                            "type": "integer",
                            "format": "int64"
                          },
                          {
                            "type": "number",
                            "format": "double"
                          },
                          {
                            "type": "string"
                          }
                        ]
                      }
                    }
                  }
                }
              ],
            }
          }
        },
        "params": {
          "type": "object",
          "properties": {
            "sentiment": {

```

```

        "type": "number",
        "format": "double",
        "default": "0.5"
    }
}
},
"examples": [
    {
        "columns": ["int_col", "float_col",
"string_col"],
        "data": [
            [3, 10.4, "abc"],
            [2, 20.4, "xyz"]
        ]
    }
],
{
    "type": "object",
    "properties": {
        "dataframe_records": {
            "type": "array",
            "items": {
                "required": ["int_col", "float_col",
"string_col"],
                "type": "object",
                "properties": {
                    "int_col": {
                        "type": "integer",
                        "format": "int64"
                    },
                    "float_col": {
                        "type": "number",
                        "format": "double"
                    },
                    "string_col": {
                        "type": "string"
                    },
                    "becx_col": {
                        "type": "object",
                        "format": "unknown"
                    }
                }
            }
        }
    },
    "params": {
        "type": "object",
        "properties": {
            "sentiment": {

```


Last updated on **May 11, 2026**

Serve multiple models to a model serving endpoint

This article describes how to programmatically configure a model serving endpoint to serve multiple models and the traffic split between them.

Serving multiple models from a single endpoint enables you to split traffic between different models to compare their performance and facilitate A/B testing. You can also serve different versions of a model at the same time, which makes experimenting with new versions easier, while keeping the current version in production.

You can serve any of the following model types on a Model Serving endpoint. You can not serve different model types in a single endpoint. For example you can not serve a custom model and an external model in the same endpoint.

- [Custom models](#)
- Generative AI models made available through Foundation Model APIs [provisioned throughput](#)
- [External models](#)

Requirements

See the [Requirements](#) for model serving endpoint creation.

To understand access control options for model serving endpoints and best practice guidance for endpoint management, see [Serving endpoint ACLs](#).

Create an endpoint and set the initial traffic split

When you create model serving endpoints using the Model Serving API or the [Model Serving UI](#), you can also set the initial traffic split for the models you want to serve on that endpoint. The following sections provide examples of setting the traffic split for multiple custom models or foundation models served on an endpoint.

Serve multiple custom models to an endpoint

The following REST API example creates a single endpoint with two custom models in Unity Catalog and sets the endpoint traffic split between those models. The served entity, `current`, hosts version 1 of `model-A` and gets 90% of the endpoint traffic, while the other served entity, `challenger`, hosts version 1 of `model-B` and gets 10% of the endpoint traffic.

Bash

POST /api/2.0/serving-endpoints

```
{
  "name": "multi-model"
  "config":
  {
    "served_entities":
    [
      {
        "name": "current",
        "entity_name": "catalog.schema.model-A",
        "entity_version": "1",
        "workload_size": "Small",
        "scale_to_zero_enabled": true
      },
      {
        "name": "challenger",
        "entity_name": "catalog.schema.model-B",
        "entity_version": "1",
        "workload_size": "Small",
        "scale_to_zero_enabled": true
      }
    ],
    "traffic_config":
```

```

{
  "routes":
  [
    {
      "served_model_name": "current",
      "traffic_percentage": "90"
    },
    {
      "served_model_name": "challenger",
      "traffic_percentage": "10"
    }
  ]
}

```

Serve multiple models to a provisioned throughput endpoint

The following REST API example creates a single Foundation Model APIs provisioned throughput endpoint with two models and sets the endpoint traffic split between those models. The endpoint named `multi-pt-model`, hosts version 2 of `meta_llama_v3_1_70b_instruct` which gets 60% of the endpoint traffic, and also hosts version 3 of `meta_llama_v3_1_8b_instruct` which gets 40% of the endpoint traffic.

Bash

POST /api/2.0/serving-endpoints

```

{
  "name": "multi-pt-model"
  "config":
  {
    "served_entities":
    [
      {
        "name": "meta_llama_v3_1_70b_instruct",
        "entity_name": "system.ai.meta_llama_v3_1_70b_instruct",
        "entity_version": "4",
        "min_provisioned_throughput": 0,
        "max_provisioned_throughput": 2400
      },
      {
        "name": "meta_llama_v3_1_8b_instruct",

```

 Ask Genie

```

        "entity_name": "system.ai.meta_llama_v3_1_8b_instruct",
        "entity_version": "4",
        "min_provisioned_throughput": 0,
        "max_provisioned_throughput": 1240
    }
],
"traffic_config":
{
    "routes":
    [
        {
            "served_model_name": "meta_llama_v3_1_8b_instruct",
            "traffic_percentage": "60"
        },
        {
            "served_model_name": "meta_llama_v3_1_70b_instruct",
            "traffic_percentage": "40"
        }
    ]
}
}

```

Serve multiple external models to an endpoint

You can also configure multiple [external models](#) in a serving endpoint as long as they all have the same task type and each model has a unique `name`. You cannot have both external models and non-external models in the same serving endpoint.

The following example creates a serving endpoint that routes 50% of the traffic to `gpt-4` provided by OpenAI and the remaining 50% to `claude-3-opus-20240229` provided by Anthropic.

Python

```

import mlflow.deployments

client = mlflow.deployments.get_deploy_client("databricks")

client.create_endpoint(
    name="mix-chat-endpoint",
    config={
        "served_entities": [
            {

```

 Ask Genie

```

        "name": "served_model_name_1",
        "external_model": {
            "name": "gpt-4",
            "provider": "openai",
            "task": "llm/v1/chat",
            "openai_config": {
                "openai_api_key": "{{secrets/my_openai_secret_scope/openai_api_key}}"
            }
        },
        {
            "name": "served_model_name_2",
            "external_model": {
                "name": "claude-3-opus-20240229",
                "provider": "anthropic",
                "task": "llm/v1/chat",
                "anthropic_config": {
                    "anthropic_api_key": "{{secrets/my_anthropic_secret_scope/anthropic_api_key}}"
                }
            }
        }
    ],
    "traffic_config": {
        "routes": [
            {"served_model_name": "served_model_name_1",
"traffic_percentage": 50},
            {"served_model_name": "served_model_name_2",
"traffic_percentage": 50}
        ]
    },
}
)

```

Update the traffic split between served models

You can also update the traffic split between served models. The following REST API example sets the served model, `current`, to get 50% of the endpoint traffic and the other model, `challenger`, to get the remaining 50% of the traffic.

You can also make this update from the **Serving** tab in the Databricks UI using the **Edit configuration** button.



Bash

```
PUT /api/2.0/serving-endpoints/{name}/config
```

```
{
  "served_entities":
  [
    {
      "name": "current",
      "entity_name": "catalog.schema.model-A",
      "entity_version": "1",
      "workload_size": "Small",
      "scale_to_zero_enabled": true
    },
    {
      "name": "challenger",
      "entity_name": "catalog.schema.model-B",
      "entity_version": "1",
      "workload_size": "Small",
      "scale_to_zero_enabled": true
    }
  ],
  "traffic_config":
  {
    "routes":
    [
      {
        "served_model_name": "current",
        "traffic_percentage": "50"
      },
      {
        "served_model_name": "challenger",
        "traffic_percentage": "50"
      }
    ]
  }
}
```

Query individual models behind an endpoint

In some scenarios, you might want to query individual models behind the endpoint.

 Ask Genie

You can do so by using:

Bash

```
POST /serving-endpoints/{endpoint-name}/served-models/{served-model-name}/invocations
```

Here the specific served model is queried. The request format is the same as querying the endpoint. While querying the individual served model, the traffic settings are ignored.

In the context of the `multi-model` endpoint example, if all requests are sent to `/serving-endpoints/multi-model/served-models/challenger/invocations`, then all requests are served by the `challenger` served model.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Monitor model quality and endpoint health

Last updated on **May 7, 2026**

Monitor model quality and endpoint health

Model Serving provides advanced tooling for monitoring the quality and health of models and their deployments. The following table is an overview of each monitoring tool available.

Tool	Description	Purpose	Access
Ephemeral service logs	Captures <code>stdout</code> and <code>stderr</code> streams from the model serving endpoint.	Useful for debugging during model deployment. Use <code>logging.warning(...)</code> or <code>logging.error(...)</code> for immediate display in the logs.	Accessible using the Logs tab in the Serving UI. Logs are streamed in real-time and can be exported through the API.
OpenTelemetry for custom model serving endpoints	Persists standard system logs, custom application logs, metrics, and traces to Unity Catalog Delta tables using OpenTelemetry	Useful for historical debugging, compliance requirements, and analyzing production issues using SQL queries.	Configure telemetry settings in the Serving UI or API when creating the endpoint. Q:  Ask Genie us

Tool	Description	Purpose	Access
	for long-term retention.		Catalog SQL or Databricks notebooks.
Build logs	Displays output from the process which automatically creates a production-ready Python environment for the model serving endpoint.	Useful for diagnosing model deployment and dependency issues.	Available upon completion of the model serving build under Build logs in the Logs tab. Logs can be exported through the API. These logs are retained for up to thirty (30) days.
Endpoint health metrics	Provides insights into infrastructure metrics like latency, request rate, error rate, CPU usage, and memory usage.	Important for understanding the performance and health of the serving infrastructure.	Available by default in the Serving UI for the last 14 days. Data can also be streamed to observability tools in real-time.
AI Gateway-enabled inference tables	Automatically logs online prediction requests and responses into Delta tables	Use this tool for monitoring and debugging model quality or responses, generating training data	Can be enabled for existing and new model serving endpoints

Tool	Description	Purpose	Access
	managed by Unity Catalog for endpoints that serve custom models, external models, or provisioned throughput workloads.	sets, or conducting compliance audits.	when enabling AI Gateway features using the Serving UI or REST API.

TIP

Use these monitoring tools to identify performance bottlenecks and optimize your endpoints. For comprehensive optimization strategies, see [Optimize Model Serving endpoints for production](#).

Additional resources

- [Track and export serving endpoint health metrics to Prometheus and Datadog](#)
- [Monitor model serving costs](#)
- [Debugging guide for Model Serving](#)
- [Optimize Model Serving endpoints for production](#)

Is this page

as this page helpful?

 Yes

 No

 Send feedback

 Ask Genie

Last updated on **May 12, 2026**

Model Serving limits and regions

This article summarizes the limitations and region availability for Databricks Model Serving and supported endpoint types.

Resource and payload limits

Model Serving imposes default limits to ensure reliable performance. If you have feedback on these limits, reach out to your Databricks account team.

The limits in this section apply to custom model and [AI agent](#) endpoints only. For Foundation Model APIs and external model resource and payload limits, see [Foundation Model APIs rate limits and quotas](#).

Custom models and AI agents

Feature	Granularity	Limit
Endpoints	Per workspace	1000. Reach out to your Databricks account team to increase.
Queries per second (QPS)	Per endpoint	300,000 using route optimization . If 1024 concurrency is not enough, reach out to your Databricks account team to increase.
Queries per second (QPS)	Per workspace	300,000 using route optimization for non-route optimized, rec

 Ask Genie

Feature	Granularity	Limit
		only for small dev use-cases.
Provisioned concurrency	Per model	1024 with custom option and route optimization . Reach out to your Databricks account team to increase.
Provisioned concurrency	Per workspace	4096. Reach out to your Databricks account team to increase.
Create/update operations	Per workspace	50 in 5 minutes.
Payload size	Per request	16 MB. For AI agent endpoints the limit is 4 MB.
Request/response size	Per request	Any request/response over 1 MB will not be logged.
Model execution duration	Per request	597 seconds
CPU endpoint model memory usage	Per model instance	4GB
GPU endpoint model memory usage	Per endpoint	Depends on GPU type
Environment variables	Per served model	50. Reach out to your Databricks account team to increase.
Overhead latency	Per request	Less than 20 milliseconds with route optimization .

Networking and security limitations Ask Genie

- Model Serving endpoints are protected by [access control](#) and respect networking-related ingress rules configured on the workspace, like IP allowlists and [PrivateLink](#).
- By default, Model Serving does not support PrivateLink to external endpoints. Support for this functionality is evaluated and implemented on a per-region basis. Reach out to your Databricks account team for more information.
- Model Serving does not provide security patches to existing model images because of the risk of destabilization to production deployments. A new model image created from a new model version will contain the latest patches. Reach out to your Databricks account team for more information.
- You can restrict outbound network access from Model Serving endpoints by configuring network policies. See [Manage network policies for serverless egress control](#).

Compliance security profile standards: CPU and GPU workloads

The following table lists the region availability and supported compliance security profile compliance standards for model serving on CPU and GPU workloads including external models.

NOTE

These compliance standards require served containers to be built in the most recent 30 days. Databricks automatically rebuilds outdated containers on your behalf. However, if this automated job fails, an event log message like the following appears and provides guidance on how to ensure your endpoints stay within compliance requirements:

```
"Databricks couldn't complete a scheduled compliance check for model
$servedModelName. This can happen if the system can't apply a required
update. To resolve, try relogging your model. If the issue persists,
contact support@databricks.com."
```

Region	Location	HIPAA	PCI-DSS	FedRAMP Moderate	IRAP	CCCS Medium (Protected B)	
ap-northeast-1	Asia Pacific (Tokyo)	✓	✓				
ap-northeast-2	Asia Pacific (Seoul)	✓	✓				
ap-south-1	Asia Pacific (Mumbai)	✓	✓				
ap-southeast-1	Asia Pacific (Singapore)	✓	✓				
ap-southeast-2	Asia Pacific (Sydney)	✓	✓		✓		
ca-central-1	Canada (Central)	✓	✓			✓	
eu-central-1	EU (Frankfurt)	✓	✓				
eu-west-1	EU (Ireland)	✓	✓				
eu-west-2	EU (London)	✓	✓				
eu-west-3	EU (Paris)						

Region	Location	HIPAA	PCI-DSS	FedRAMP Moderate	IRAP	CCCS Medium (Protected B)	
sa-east-1	South America (Sao Paulo)	✓	✓				
us-east-1	US East (Northern Virginia)	✓	✓	✓			
us-east-2	US East (Ohio)	✓	✓	✓			
us-gov-west-1	US Gov West (Pendleton)						
us-west-1	US West (Oregon)						
us-west-2	US West (Oregon)	✓	✓	✓			

Compliance security profile standards: Foundation Model APIs workloads

The table lists the supported compliance security profile compliance standards for the following Foundation Model APIs workloads:

- Provisioned throughput
- Pay-per-token
- Batch inference using AI Functions and Databricks-hosted models

NOTE

These compliance standards require served containers to be built in the most recent 30 days. Databricks automatically rebuilds outdated containers on your behalf. However, if this automated job fails, an event log message like the following appears and provides guidance on how to ensure your endpoints stay within compliance requirements:

```
"Databricks couldn't complete a scheduled compliance check for model $servedModelName. This can happen if the system can't apply a required update. To resolve, try relogging your model. If the issue persists, contact support@databricks.com."
```

Region	Location	HIPAA	PCI-DSS	FedRAMP Moderate	IRAP	CCCS Medium (Protected B)	
ap-northeast-1	Asia Pacific (Tokyo)	✓	✓				
ap-northeast-2	Asia Pacific (Seoul)	✓	✓				
ap-south-1	Asia Pacific (Mumbai)	✓	✓				
ap-southeast-1	Asia Pacific (Singapore)	✓	✓				
ap-southeast-2	Asia Pacific (Sydney)	✓	✓		✓		

 Ask Genie

Region	Location	HIPAA	PCI-DSS	FedRAMP Moderate	IRAP	CCCS Medium (Protected B)	
ca-central-1	Canada (Central)	✓	✓			✓	
eu-central-1	EU (Frankfurt)	✓	✓				
eu-west-1	EU (Ireland)	✓	✓				
eu-west-2	EU (London)	✓	✓				✓
eu-west-3	EU (Paris)						
sa-east-1	South America (Sao Paulo)	✓	✓				
us-east-1	US East (Northern Virginia)	✓	✓	✓			
us-east-2	US East (Ohio)	✓	✓	✓			
us-gov-west-1	US Gov West (Pendleton)						
us-west-1	US West (Oregon)						

Ask Genie

Region	Location	HIPAA	PCI-DSS	FedRAMP Moderate	IRAP	CCCS Medium (Protected B)	
us-west-2	US West (Oregon)	✓	✓	✓			

* Some models require [cross geography routing](#) for provisioned throughput and therefore are not UK Cyber Essentials Plus compliant. Reach out to your Databricks account team for more information.

Foundation Model APIs limits

For detailed information about Foundation Model APIs, including resource and payload limits for foundation and external models, see [Foundation Model APIs rate limits and quotas](#).

Region availability

NOTE

If you require an endpoint in an unsupported region, reach out to your Databricks account team.

If your workspace is deployed in a region that supports model serving but is served by a [control plane](#) in an unsupported region, the workspace does not support model serving. If you attempt to use model serving in such a workspace, you will see in an error message stating that your workspace is not supported. Reach out to your Databricks account team for more information.

See [Model serving features availability](#) for more information on regional availability of each Model Serving feature.

For Databricks-hosted foundation model region availability, see [Foundation models hosted on Databricks](#).

Is this page
as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **May 13, 2026**

Debug model serving timeouts

This article describes the different timeouts you might encounter when using model serving endpoints and how to handle them. It covers model deployment timeouts, server-side timeouts, and client-side timeouts.

Model deployment timeouts

When you deploy a model or update an existing deployment using Model Serving, the process might time out for a number of reasons. The **Events** tab of the model serving endpoint page records timeout messages. Search on **"timed out"** to find them.

NOTE

The deployment process times out if the container build and model deployment exceed a certain duration that is dependent on the endpoint workload configuration. Check your configuration before deploying and compare it to previous successful deployments.

The container build has no hard limit, but retries up to 3 times. The deployment after the container is built will wait up to 30 minutes for CPU workloads, 60 minutes for GPU small or medium workloads, and 120 minutes for GPU large workloads before timing out.

text-extract-demo-v1

Permissions

Query endpoint

Serving endpoint state: ⊗ Not ready (Update failed)

Created by: 8d7f57ac-b30a-44d0-83bb-8a5e124ff05f

URL: <https://yipitdata-product.cloud.databricks.com/serving-endpoints/text-extract-demo-v1/invocations>

Tags: [🔗](#)

Failed configuration

Active

⊗ Failed

Edit configuration

Model	Version	Name	State	Compute	Traffic (%)
■ text_extract_demo	Version 1	text_extract_demo_1	⊗ Aborted	GPU Medium (A10G), Small 4 concurrent requests (24 DBU)	100

Metrics

Events

Logs

Timestamp	Event type	Served model name	Message
2023-08-17 10:38:13	ENDPOINT_UPDATE_EVENT		Endpoint update failed for endpoint 'text-extract-demo-v1', config version 2.
2023-08-17 10:38:13	SERVED_MODEL_SERVICE_EVENT	text_extract_demo_1	Served model creation aborted for served model 'text_extract_demo_1', config version 2, since the update timed out.
2023-08-17 09:52:17	SERVED_MODEL_CREATION_EVENT	text_extract_demo_1	Served model created for served model 'text_extract_demo_1', config version 2.
2023-08-17 09:52:16	ENDPOINT_UPDATE_EVENT		Endpoint updated by slarson@yipitdata.com.
2023-08-16 16:44:24	ENDPOINT_UPDATE_EVENT		Endpoint update failed for endpoint 'text-extract-demo-v1', config version 1.
2023-08-16 16:44:24	SERVED_MODEL_SERVICE_EVENT	text_extract_demo_1	Served model creation aborted for served model 'text_extract_demo_1', config version 1, since the update timed out.

If you find a **"timed out"** message, navigate to the **Logs** tab and examine the build logs to determine the cause. Examples include library dependency issues, resource constraints, configuration issues and so on.

Metrics

Events

Logs

main.default. (Version 1)

Service logs

Build logs

See [Debug after container build failure](#).

Server-side timeouts

If your endpoint is healthy according to the **Events** and **Logs** tabs of your serving endpoint, but you experience timeouts when you make calls to the endpoint, the timeout might be on the server-side. The default timeout varies depending on the type of model serving endpoint. The following table shows the default server-side timeouts for requests sent to model serving endpoints.

Endpoint type	Request timeout limit (seconds)	Notes
CPU or GPU serving endpoints	Default 597	This limit cannot be increased.
Foundation model serving endpoints	Default 597	This limit cannot be increased.

To determine if you experienced a server-side timeout, see if your requests are timing out before or after the limits listed above.

- If your request consistently fails at the limit, then it's likely a server-side timeout.
- If your request fails earlier than the limit, it might be due to configuration issues.
 - Check the service logs to determine if there are any other errors.
 - Confirm that the model has worked locally, like from a notebook, or on previous requests on earlier versions.

Client-side timeouts: MLflow configuration

Client-side timeouts typically return error messages that say ***"timed out"*** or **4xx Bad Request**. Common causes of these timeouts are from MLflow environment variables configurations. The following are the most common MLflow environment variables for timeouts. For the full list of timeout variables, see the [mlflow.environment_variables documentation](#).

- **MLFLOW_HTTP_REQUEST_TIMEOUT**: Specifies the timeout in seconds for MLflow HTTP requests. Default timeout 120 seconds.
- **MLFLOW_HTTP_REQUEST_MAX_RETRIES**: Specifies the maximum number of retries with exponential backoff for MLflow HTTP requests. Default is 7 seconds.

NOTE

The HTTP request timeouts on the client-side are set to 120 seconds, which differs from the server-side's default timeout of 597 seconds for CPU and GPU

 Ask Genie

serving endpoints. Adjust the MLflow environment variables accordingly if you expect your workload to exceed the 120-second client side timeout.

Do either of the following to determine if a timeout is caused by an MLflow environment variable configuration:

- Test the model locally using sample inputs, like in a notebook, to confirm that it works as expected before you register the model and deploy it.
 - Examine the time it takes to process the requests.
 - If requests take longer than the default timeouts for MLflow environment variables or you get a **"timed out"** message in the notebook. Example **"timed out"** message:
 - Timed out while evaluating the model. Verify that the model evaluates within the timeout.
- Test the model serving endpoint using POST requests.
 - Check the **Service Logs** for your endpoint or the inference tables if you enabled them.
 - For inference table schema details, see [Unity AI Gateway-enabled inference table schema](#).

Configure MLflow environment variables

Configure MLflow environment variables using the Serving UI or programmatically using Python.

Serving UI Python

You can configure environment variables for a model deployment

1. Select the endpoint that you want to configure an environment variable for.
2. On the endpoint's page, select **Edit** on the top right.
3. In **"Entity Details"**, expand **Advanced configuration** to add the relevant MLflow timeout environment variable.

See [Add plain text environment variables](#).

Client-side timeouts: Third party client APIs

Client-side timeouts typically return error messages that say **"timed out"** or **4xx Bad Request**. Similar to MLflow configurations, third party client APIs can cause client-side timeouts depending on their configuration. These can impact model serving endpoints that consist of pipelines that use these third party client APIs. See [custom PyFunc models](#) and [PyFunc custom schema agents](#).

Similar to the MLflow configuration debugging instructions, do the following to determine if a timeout is caused by 3rd-party client APIs used in your model pipeline:

- Test the model locally with sample inputs in a notebook.
 - If you see a **"timed out"** message in the notebook, adjust any relevant parameters for the third party client's timeout window.
 - Example **"timed out"** message: `APITimeoutError: Request timed out.`
- Test the model serving endpoint using POST requests.
 - Check the **Service Logs** for your endpoint or the inference tables if you enabled them.
 - For inference table schema details, see [Unity AI Gateway-enabled inference table schema](#).

OpenAI client example

When you establish an [OpenAI client](#), you can configure the `timeout` parameter to change the maximum time before a request times out on the client-side. The default and maximum timeout for an OpenAI client is [10 minutes](#).

The following example highlights how to configure a 3rd-party client APIs timeout.

Python

```
%pip install openai
dbutils.library.restartPython()

from openai import OpenAI
import os
```

 Ask Genie

```
# How to get your Databricks token: https://docs.databricks.com/en/dev-
tools/auth/pat.html
DATABRICKS_TOKEN = os.environ.get('DATABRICKS_TOKEN')
client = OpenAI(
    timeout=10, # Number of seconds before client times out
    api_key=DATABRICKS_TOKEN,
    base_url="<WORKSPACE_URL>/serving-endpoints"
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "You are an AI assistant"
        },
        {
            "role": "user",
            "content": "Tell me about Large Language Models."
        }
    ],
    model="model_name",
    max_tokens=256
)
```

NOTE

For the OpenAI client, you can get around the maximum timeout window by enabling [streaming](#).

Other timeouts

Idle endpoints warming up

If an endpoint is [scaled to 0](#) and it receives a request that warms it up, it could potentially lead to a client-side timeout if it takes too long to warm up. This can be a cause of timeouts in pipelines that leverage steps like calls to provisioned throughput endpoints or vector search indices, as mentioned above.

Connection timeout

Connection timeouts are related to the time a client waits to establish a connection with the server. If the connection is not established within this time, the client cancels the attempt. It's important to be aware of the clients used in your model pipeline and check the service logs and inference tables of the Model Serving endpoint for any connection timeouts. The messaging varies by service.

- For example, a [SocketTimeout](#) (for a service reading/writing to a SQL endpoint over a JDBC connection) may look like the following:
 - `jdbc:spark://<server-hostname>:443;HttpPath=<http-path>;TransportMode=http;SSL=1[;property=value[;property=value]];SocketTimeout=300`
- To find these, look for error messages containing the term ***"timed out"*** or ***"timeout"***.

Rate limits

Multiple requests made over the rate limit of an endpoint might lead to failure for additional requests. See [Resource and payload limits](#) for rate limits based on endpoint types. For third party clients, Databricks recommends that you review the documentation of the third party client you are using.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Migrate to Model Serving

This article demonstrates how to enable Model Serving in your workspace and switch your models to the [Model Serving](#) experience built on serverless compute.

❗ IMPORTANT

Starting August 22, 2025, customers will no longer be able to create new serving endpoints using the Legacy MLflow Model Serving experience. On September 15, 2025, the legacy experience will reach end of life and all existing endpoints using this service can no longer be used.

Requirements

- Registered model in the MLflow Model Registry.
- Permissions on the registered models as described in the [access control guide](#).
- [Enable serverless compute on your workspace](#).

Significant changes

- In Model Serving, the format of the request to the endpoint and the response from the endpoint are slightly different from Legacy MLflow Model Serving. See [Scoring a model endpoint](#) for details on the new format protocol.
- In Model Serving, the endpoint URL includes `serving-endpoints` instead of `model`.
- Model Serving includes full support for [managing resources with API workflows](#).
- Model Serving is production-ready and backed by the Databricks SLA.

Identify serving endpoints that use Legacy MLflow Model Serving

To identify model serving endpoints that use Legacy MLflow Model Serving:

1. Navigate to the **Models** UI in your workspace.
2. Select the **Workspace Model Registry** filter.
3. Select the **Legacy serving enabled only** filter.

Migrate Legacy MLflow Model Serving served models to Model Serving

You can create a Model Serving endpoint and flexibly transition model serving workflows without disabling [Legacy MLflow Model Serving](#).

The following steps show how to accomplish this with the UI. For each model on which you have Legacy MLflow Model Serving enabled:

1. Register your model to Unity Catalog.
2. Navigate to **Serving endpoints** on the sidebar of your machine learning workspace.
3. Follow the workflow described in [Create custom model serving endpoints](#) on how to create a serving endpoint with your model.
4. Transition your application to use the new URL provided by the serving endpoint to query the model, along with the new scoring format.
5. When your models are transitioned over, you can navigate to **Models** on the sidebar of your machine learning workspace.
6. Select the model for which you want to disable Legacy MLflow Model Serving.
7. On the **Serving** tab, select **Stop**.
8. A message appears to confirm. Select **Stop Serving**.

Migrate deployed model versions to Model Serving

In previous versions of the Model Serving functionality, the serving endpoint was created based on the stage of the registered model version: `Staging` or `Production`. To migrate your served models from that experience, you can replicate that behavior in the new Model Serving experience.

This section demonstrates how to create separate model serving endpoints for `Staging` model versions and `Production` model versions. The following steps show how to accomplish this with the serving endpoints API for each of your served models.

In the example, the registered model name `modelA` has version 1 in the model stage `Production` and version 2 in the model stage `Staging`.

1. Create two endpoints for your registered model, one for `Staging` model versions and another for `Production` model versions.

For `Staging` model versions:

Bash

```
POST /api/2.0/serving-endpoints
{
  "name": "modelA-Staging"
  "config":
  {
    "served_entities":
    [
      {
        "entity_name": "model-A",
        "entity_version": "2", // Staging Model Version
        "workload_size": "Small",
        "scale_to_zero_enabled": true
      },
    ],
  },
}
```

For `Production` model versions:

Bash

```
POST /api/2.0/serving-endpoints
{
```

 Ask Genie

```

    "name": "modelA-Production"
    "config":
    {
        "served_entities":
        [
            {
                "entity_name": "model-A",
                "entity_version": "1",    // Production Model Version
                "workload_size": "Small",
                "scale_to_zero_enabled": true
            },
        ],
    },
}

```

2. Verify the status of the endpoints.

For Staging endpoint: `GET /api/2.0/serving-endpoints/modelA-Staging`

For Production endpoint: `GET /api/2.0/serving-endpoints/modelA-Production`

3. Once the endpoints are ready, query the endpoint using:

For Staging endpoint: `POST /serving-endpoints/modelA-Staging/invocations`

For Production endpoint: `POST /serving-endpoints/modelA-Production/invocations`

4. Update the endpoint based on model version transitions.

In the scenario where a new model version 3 is created, you can have the model version 2 transition to `Production`, while model version 3 can transition to `Staging` and model version 1 is `Archived`. These changes can be reflected in separate model serving endpoints as follows:

For the `Staging` endpoint, update the endpoint to use the new model version in `Staging`.

Bash

```

PUT /api/2.0/serving-endpoints/modelA-Staging/config
{
    "served_entities":

```

 Ask Genie

```
[
  {
    "entity_name":"model-A",
    "entity_version":"3", // New Staging model version
    "workload_size":"Small",
    "scale_to_zero_enabled":true
  },
],
]
```

For **Production** endpoint, update the endpoint to use the new model version in **Production**.

Bash

```
PUT /api/2.0/serving-endpoints/modelA-Production/config
{
  "served_entities":
  [
    {
      "entity_name":"model-A",
      "entity_version":"2", // New Production model version
      "workload_size":"Small",
      "scale_to_zero_enabled":true
    },
  ],
}
```

Additional resources

- [Create Model Serving endpoints](#)

Is this page

as this page helpful?

☐ Yes

☐ No

[Ask Genie](#)

☐ Send feedback